

Réseaux de Neurones Récurrents



MonCoachData



Deep Learning

- Nous avons utilisé les réseaux de neurones pour résoudre des problèmes de classification et de régression, mais nous n'avons pas encore vu comment les réseaux de neurones peuvent traiter les informations de séquence.





Deep Learning

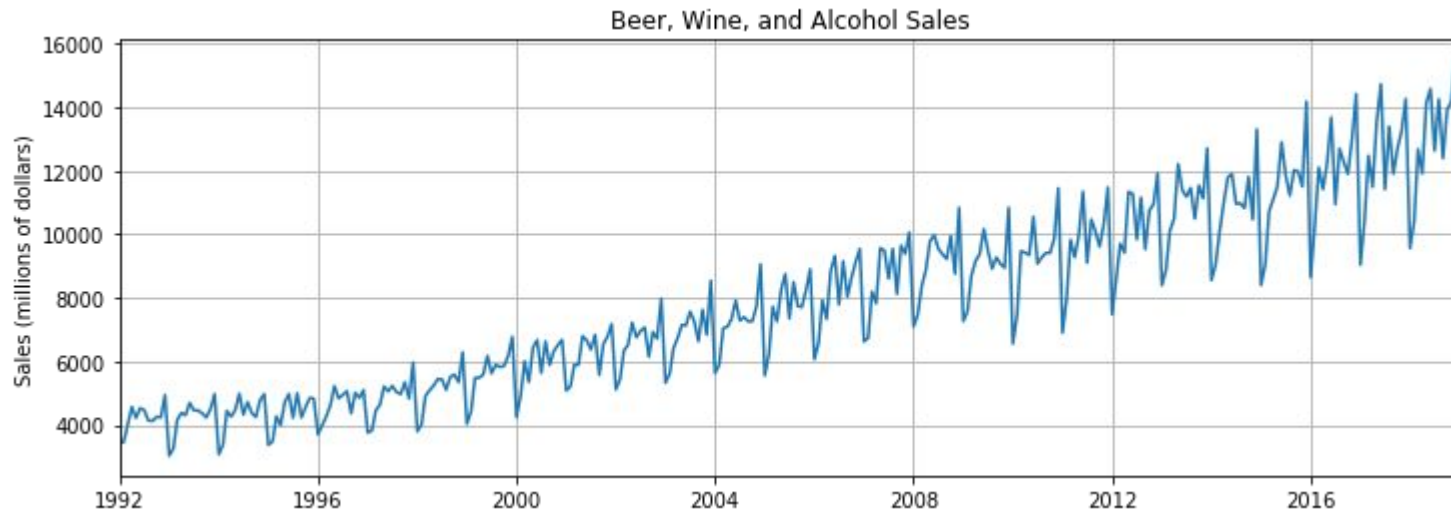
- Tout comme les CNNs étaient plus efficaces pour les données d'images 2D, les RNNs sont plus efficaces pour les données de séquence (par exemple, données de ventes horodatées, séquence de texte, données de battements de coeur, etc...)





Deep Learning

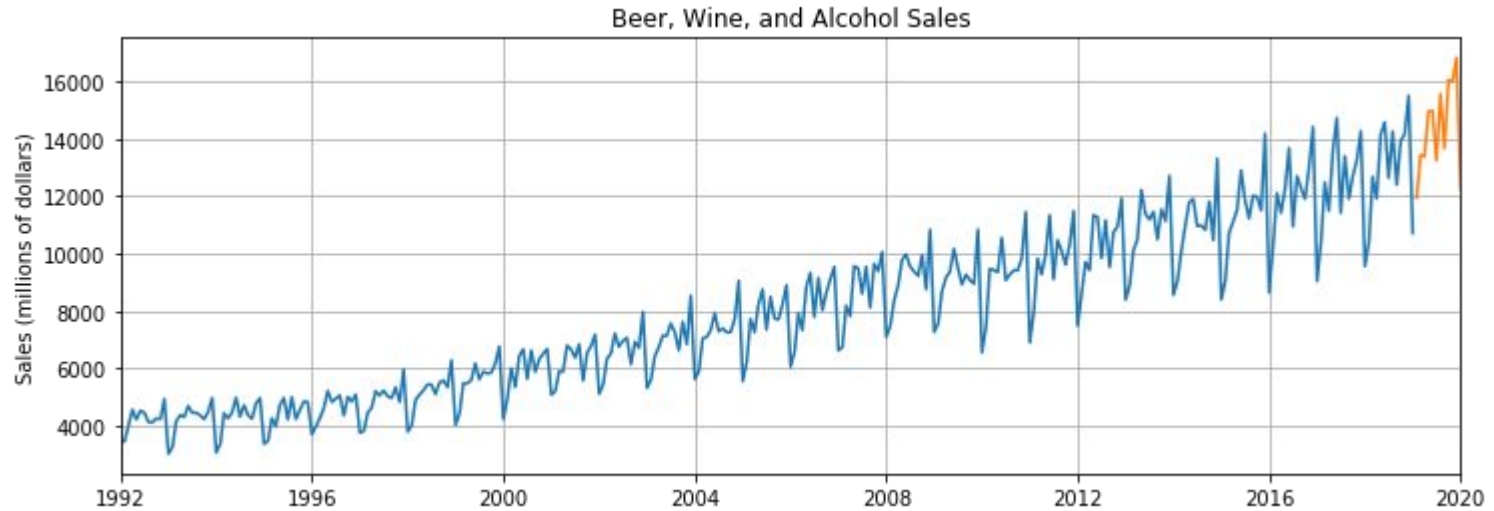
- Time Series





Deep Learning

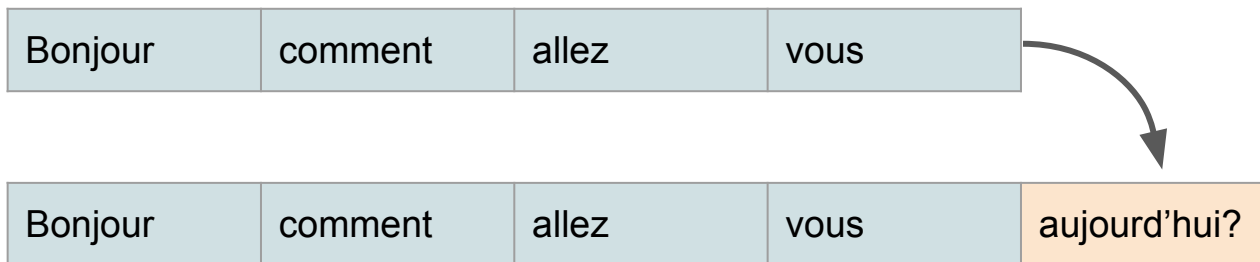
- Time Series





Deep Learning

- Séquences





Deep Learning

- Aperçu de la section
 - Théorie d'un RNN
 - Théorie des LSTMs et GRU
 - Implémentation basique d'un RNN
 - Time Series avec un RNN
 - Exercice et Solution



Théorie sur les Réseaux de Neurones Récurrents



MonCoachData



Deep Learning

- Exemples de Séquences
 - Données de Time Series (Ventes)
 - Phrases
 - Audio
 - Trajectoires de Voiture
 - Musique





Deep Learning

- Imaginons une séquence :
 - [1,2,3,4,5,6]
- Seriez-vous en mesure de prédire une séquence similaire décalée d'une unité de temps dans le futur ?
 - [2,3,4,5,6,7]





Deep Learning

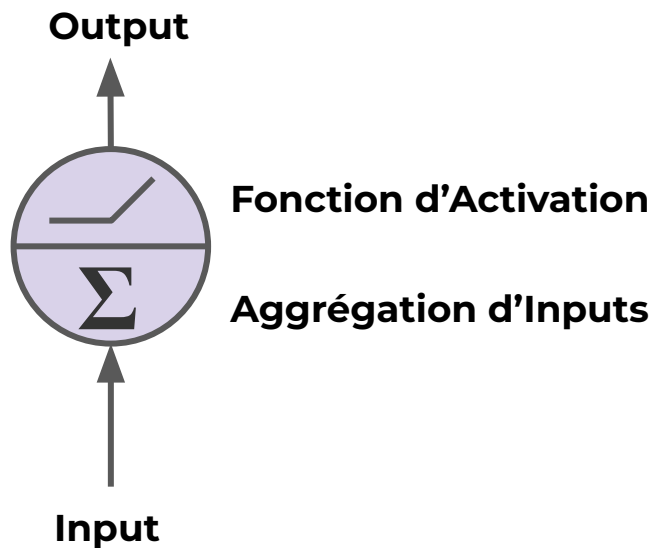
- Pour le faire correctement, nous devons d'une manière ou d'une autre faire "savoir" au neurone l'historique de ses sorties antérieures.
- Une façon simple de le faire est de réinjecter ses résultats en tant qu'inputs !





Deep Learning

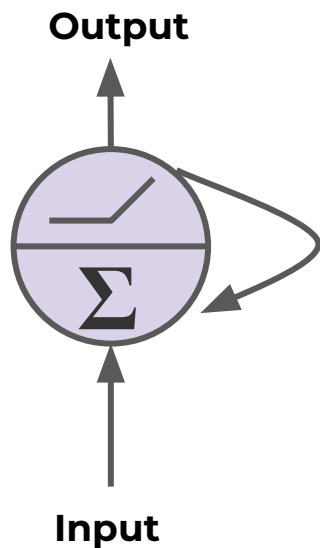
- Neurone normal dans le réseau Feed Forward





Deep Learning

- Neurone Récurrent



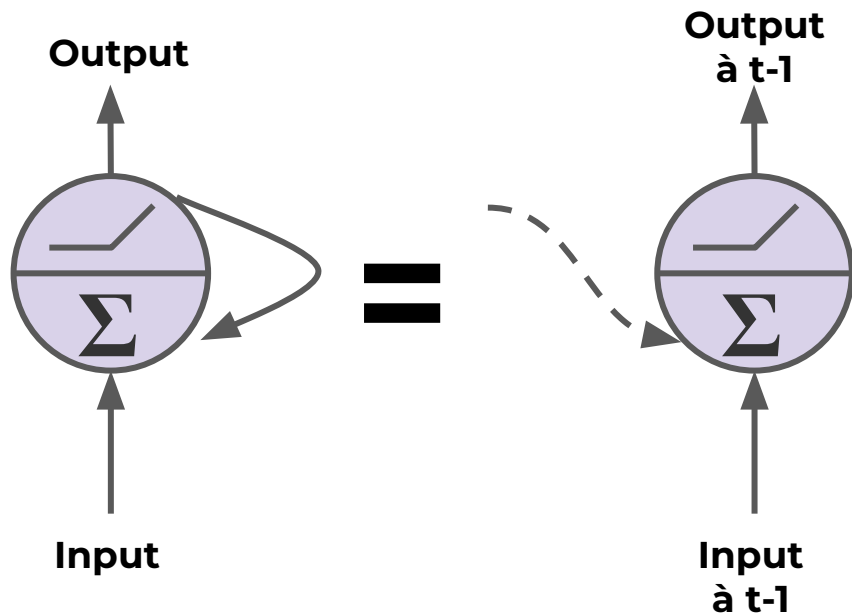
- Renvoie la sortie (output) à elle-même !
- Voyons à quoi cela ressemble dans le temps !





Deep Learning

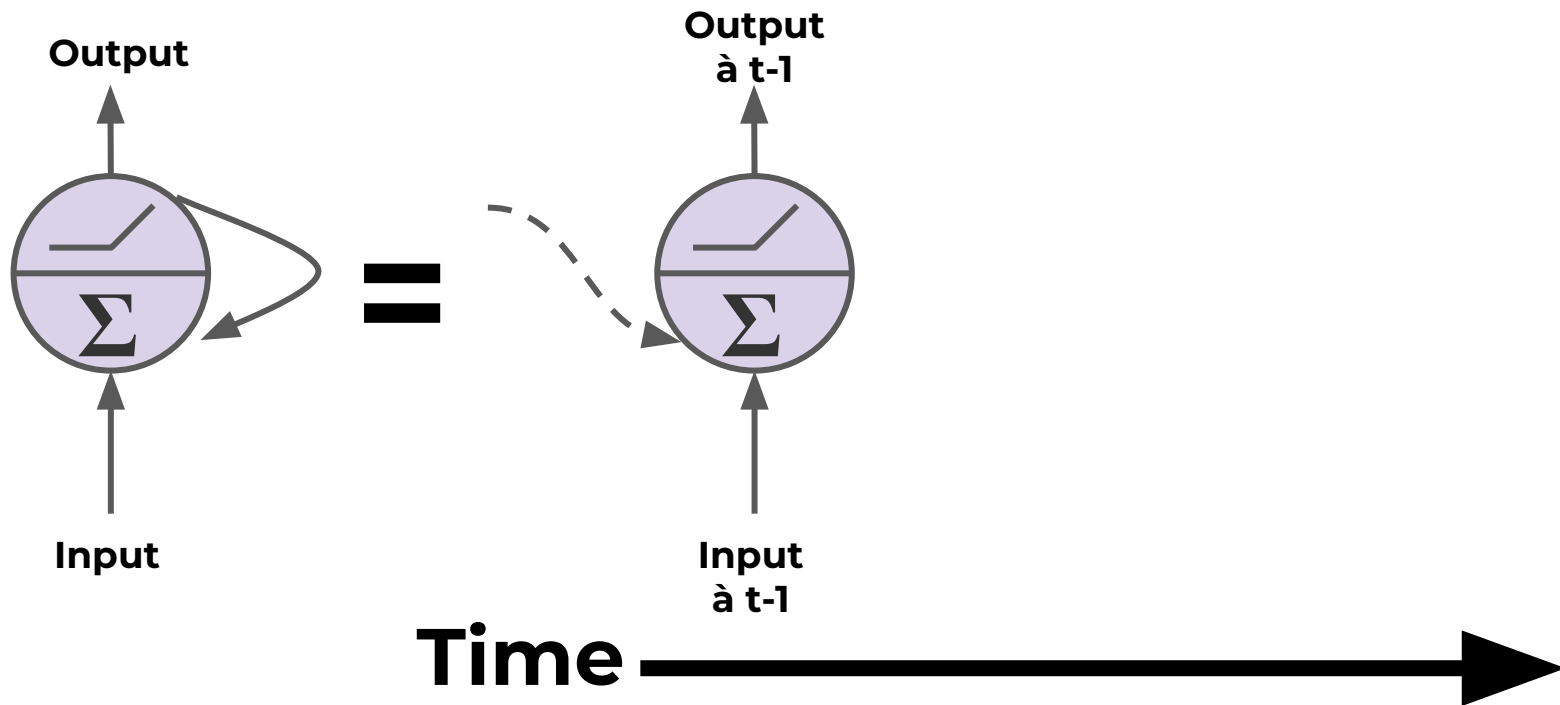
- Neurone Récurrent





Deep Learning

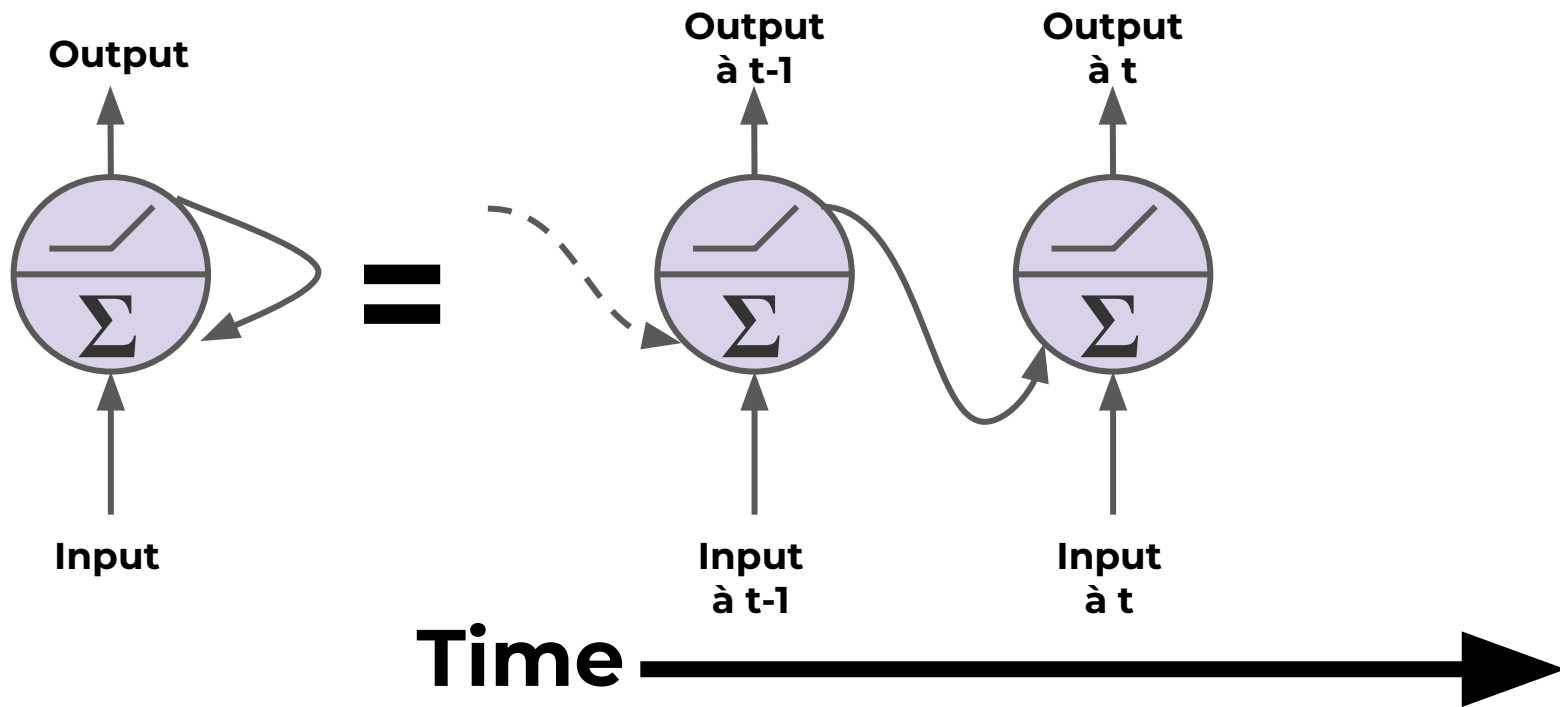
- Neurone Récurrent





Deep Learning

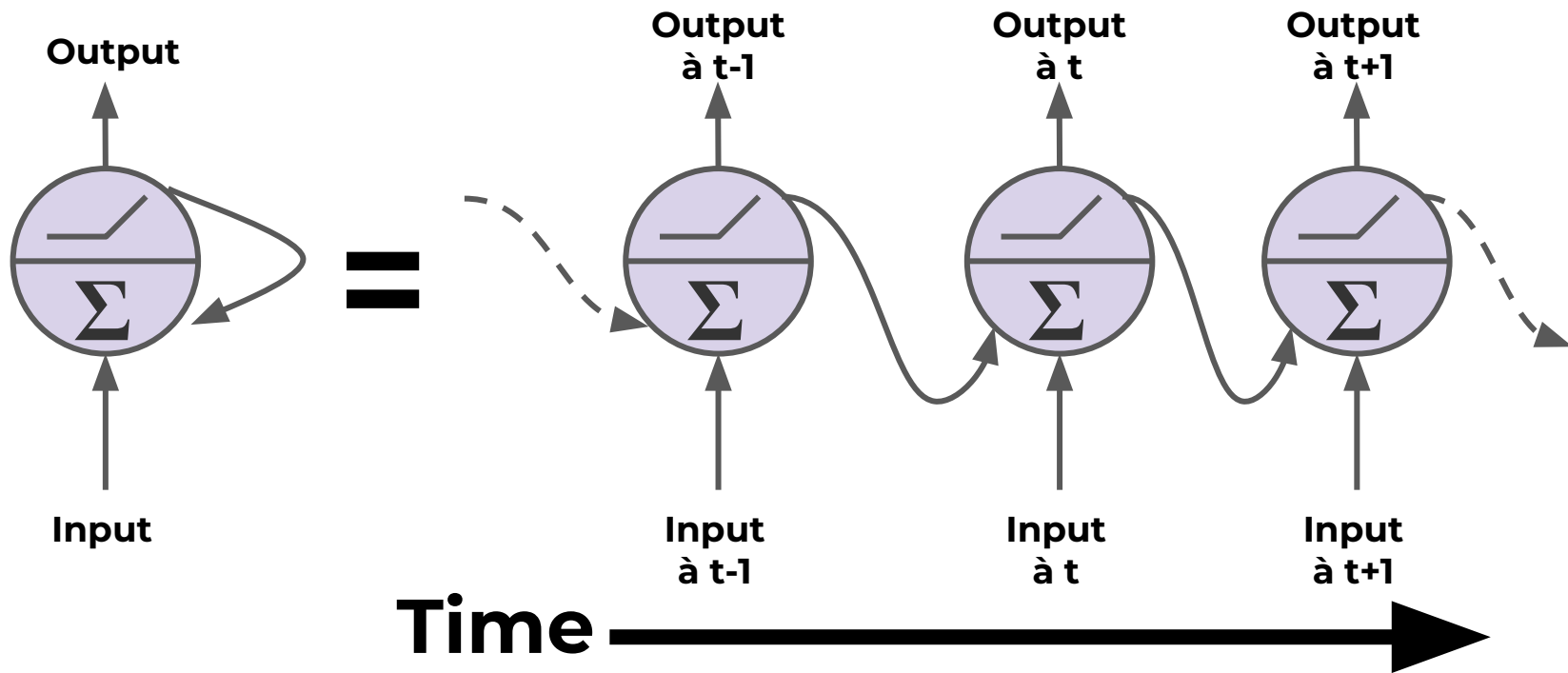
- Neurone Récurrent





Deep Learning

- Neurone Récurrent





Deep Learning

- Les cellules qui sont une fonction des entrées (inputs) au cours des unités de temps précédents sont également connues sous le nom de *cellules de mémoire*.
- Les RNNs sont également flexibles dans leurs entrées (inputs) et sorties (outputs), tant pour les séquences que pour les valeurs de vecteurs uniques.





Deep Learning

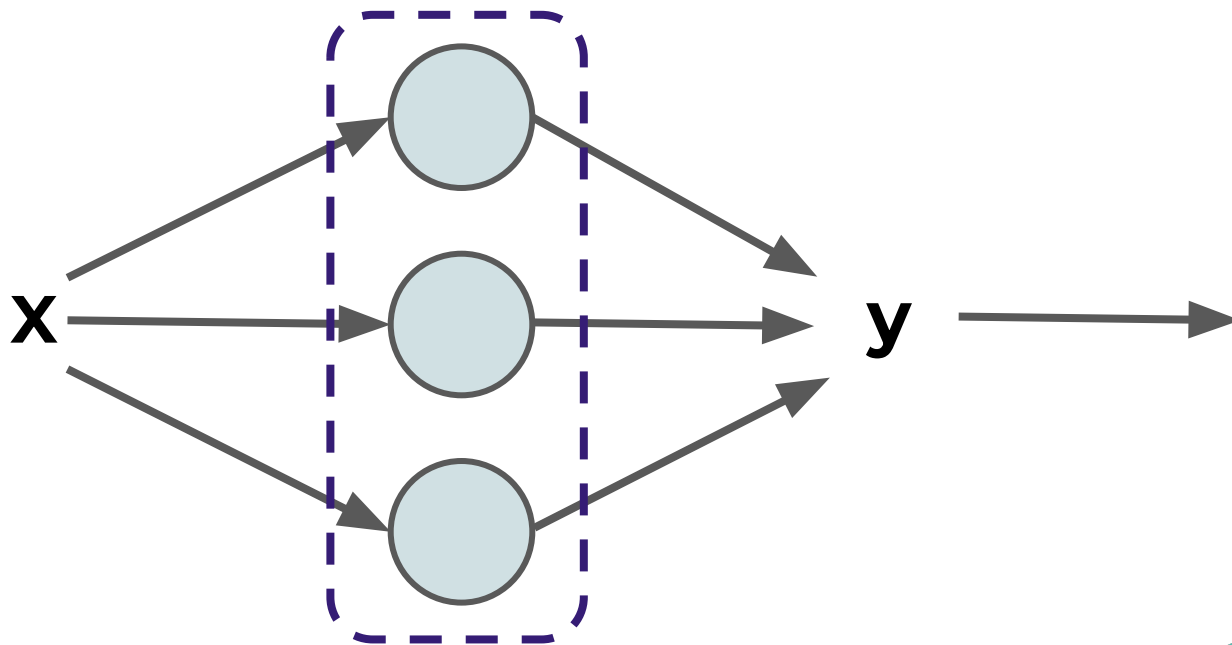
- Nous pouvons également créer des couches entières de neurones récurrents...





Deep Learning

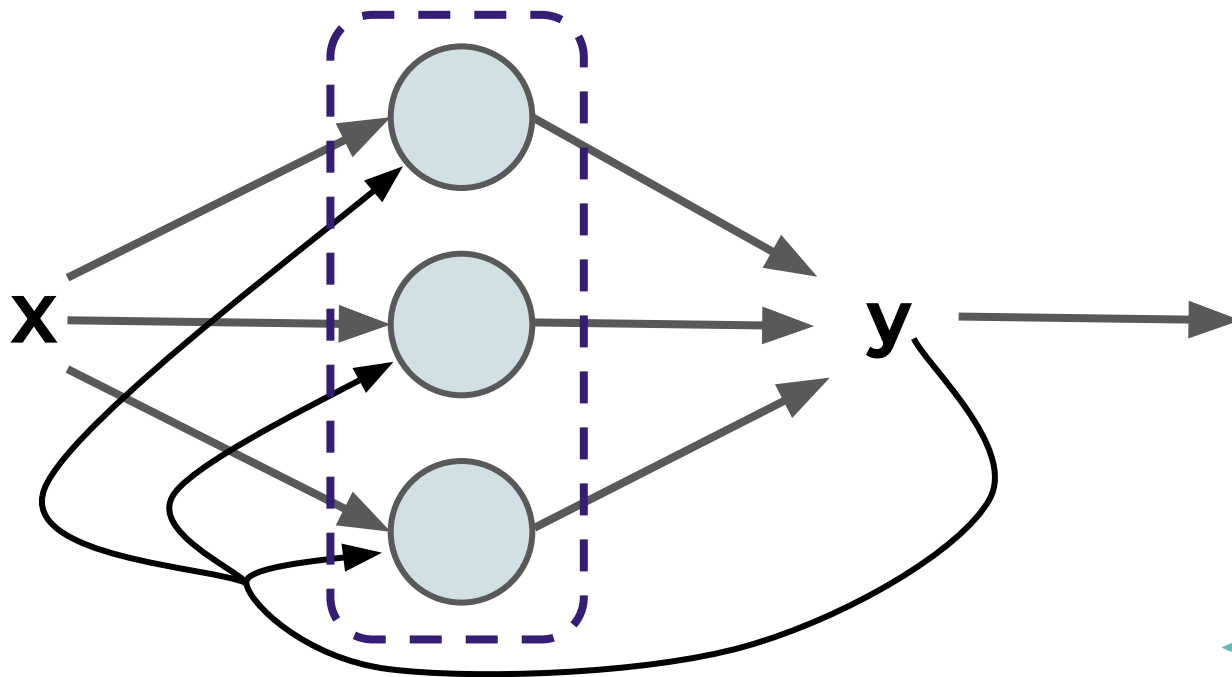
- ANN Couche avec 3 Neurones :





Deep Learning

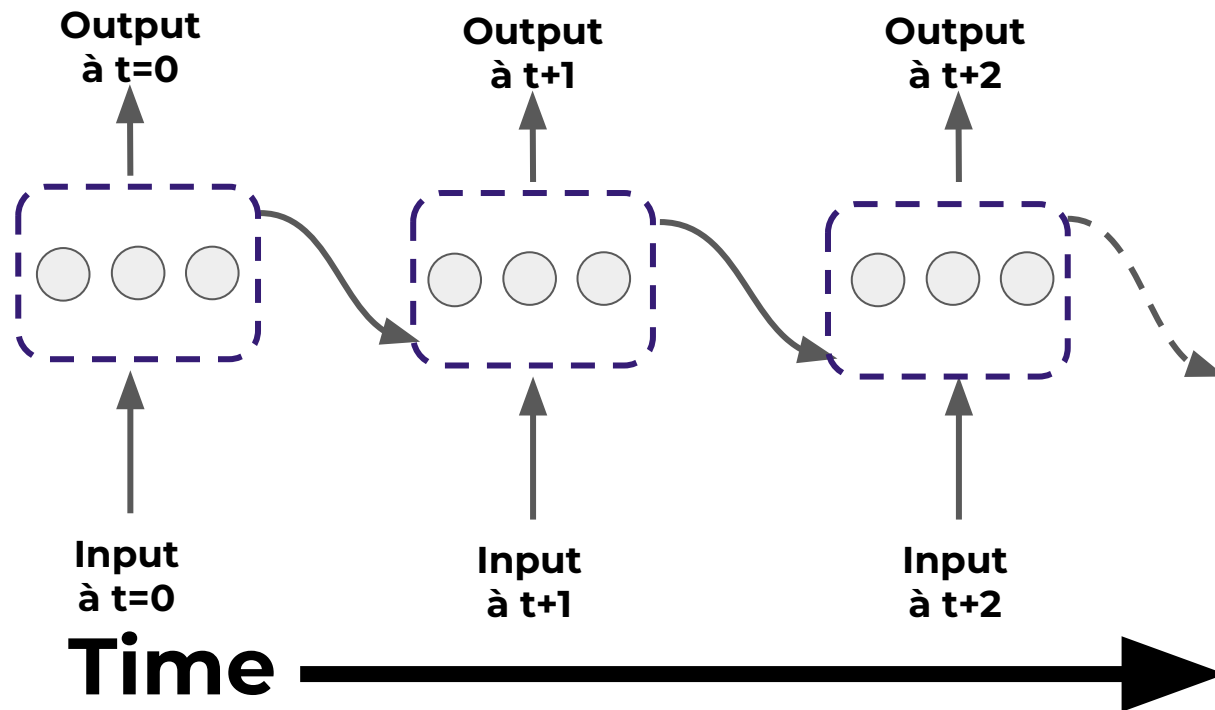
- RNN Couche avec 3 Neurones :





Deep Learning

- “Unrolled” layer





Deep Learning

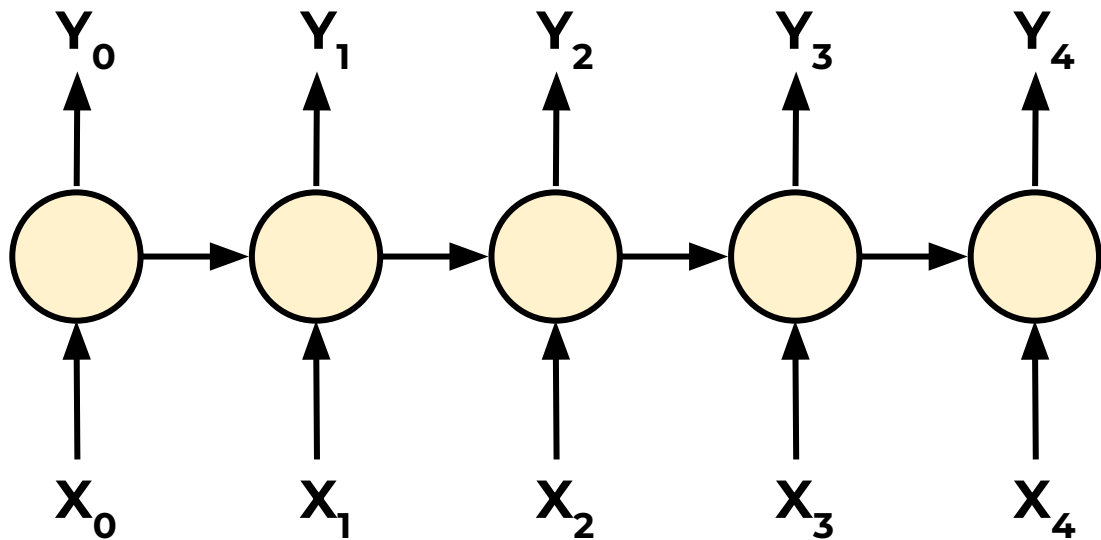
- Les RNNs sont également très flexibles dans leurs entrées (inputs) et sorties (outputs).
- Voyons quelques exemples.





Deep Learning

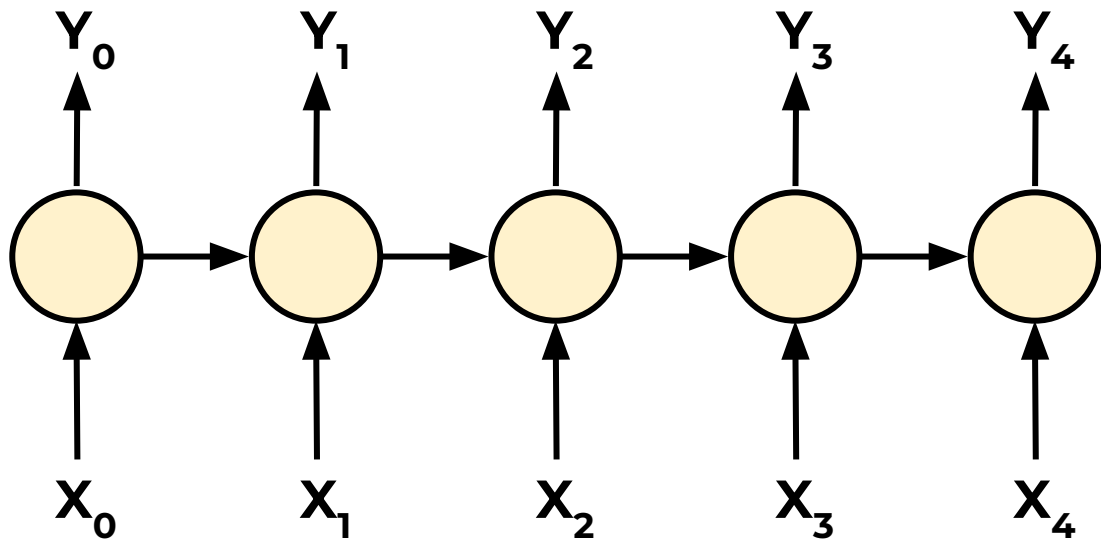
- Séquence à Séquence (Many to Many)





Deep Learning

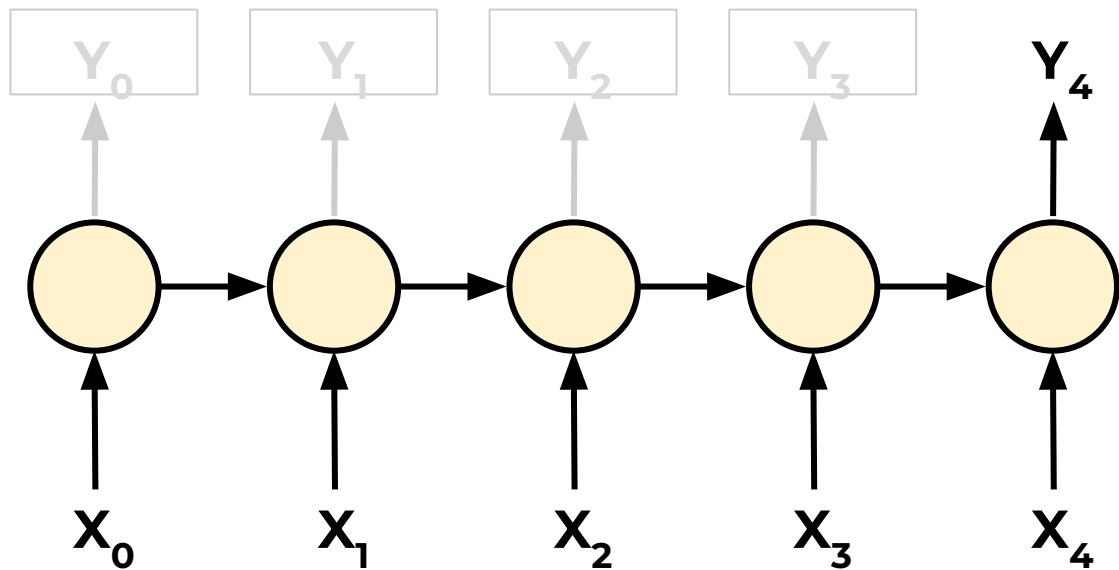
- Compte tenu des 5 mots précédents, prévoir les 5 suivants





Deep Learning

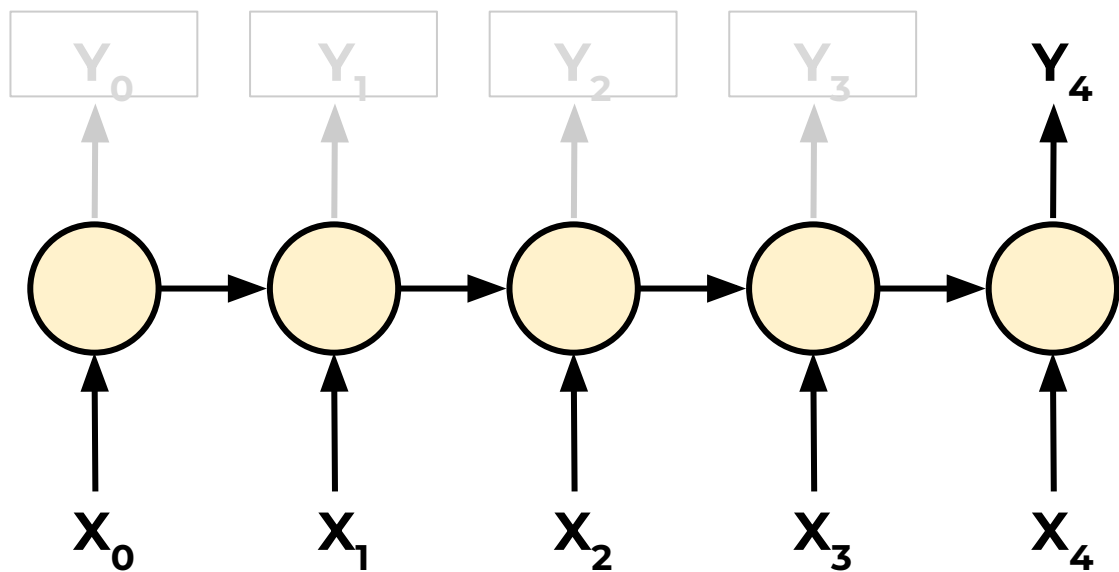
- Séquence à Vecteur (Many to One)





Deep Learning

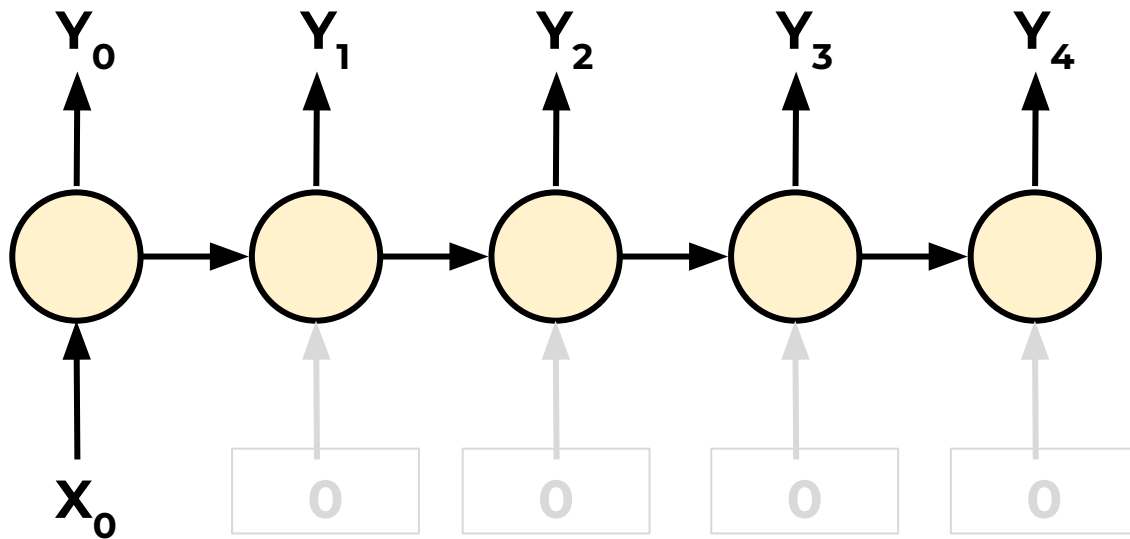
- Compte tenu des 5 mots précédents, prédire le mot suivant





Deep Learning

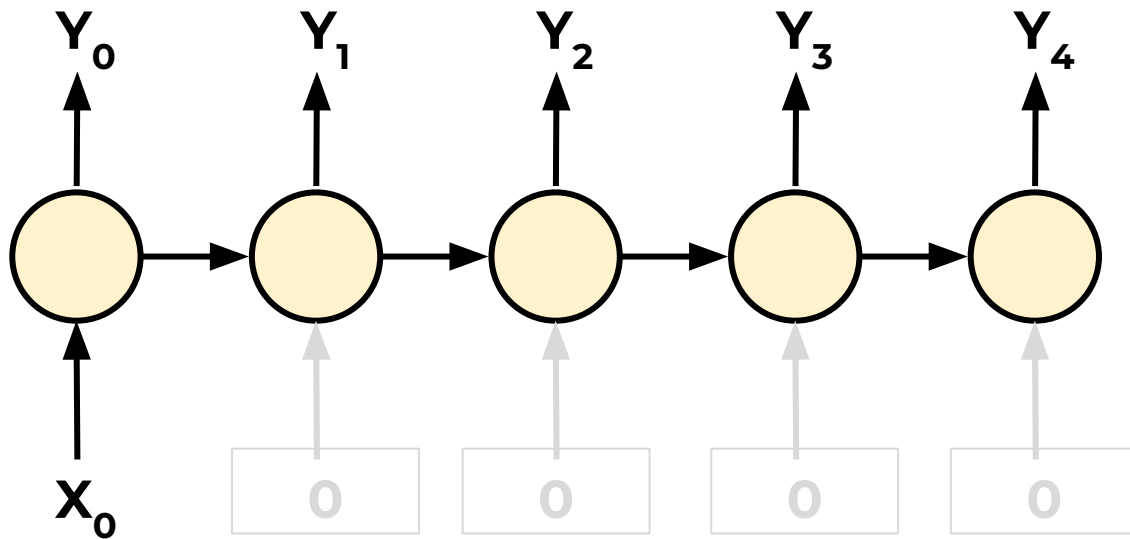
- Vecteur vers Séquence (One to Many)





Deep Learning

- Compte tenu d'1 mot, prédire les 5 mots suivants





Deep Learning

- Un RNN de base présente un inconvénient majeur, on ne "se souvient" vraiment que de la sortie précédente.
- Ce serait bien si nous pouvions suivre l'histoire à long terme, et pas seulement à court terme.





Deep Learning

- Une autre question qui se pose pendant la formation est celle du "vanishing gradient".
- Examinons plus en détail les vanishing gradients avant de passer à la discussion des LSTMs (Long Short Term Memory Units).



Exploding et Vanishing Gradients



MonCoachData



Deep Learning

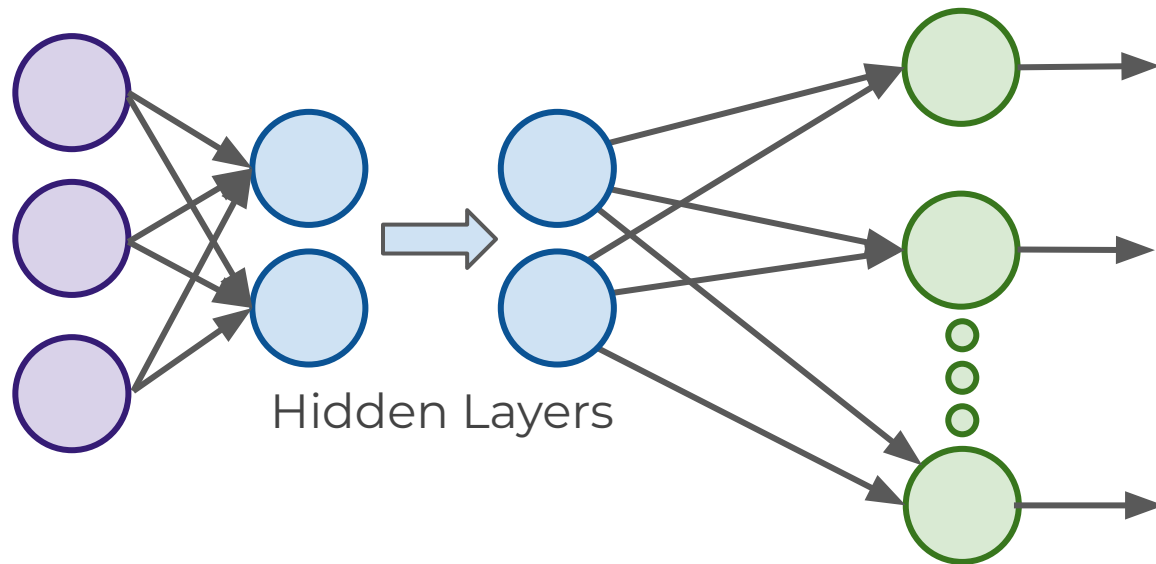
- À mesure que nos réseaux s'approfondissent et se complexifient, deux problèmes se posent :
 - Exploding Gradients
 - Vanishing Gradients
- Rappelons que le gradient est utilisé dans nos calculs pour ajuster les poids et les biais dans notre réseau.





Deep Learning

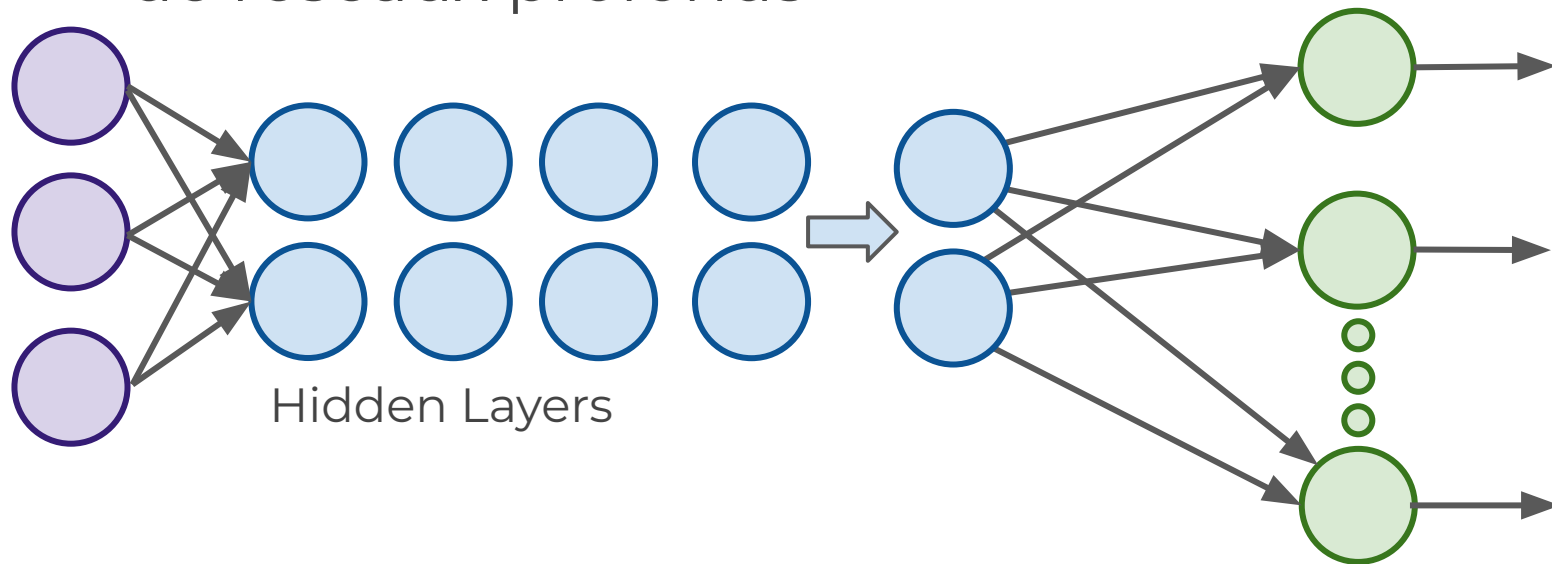
- Pensons à un réseau de base





Deep Learning

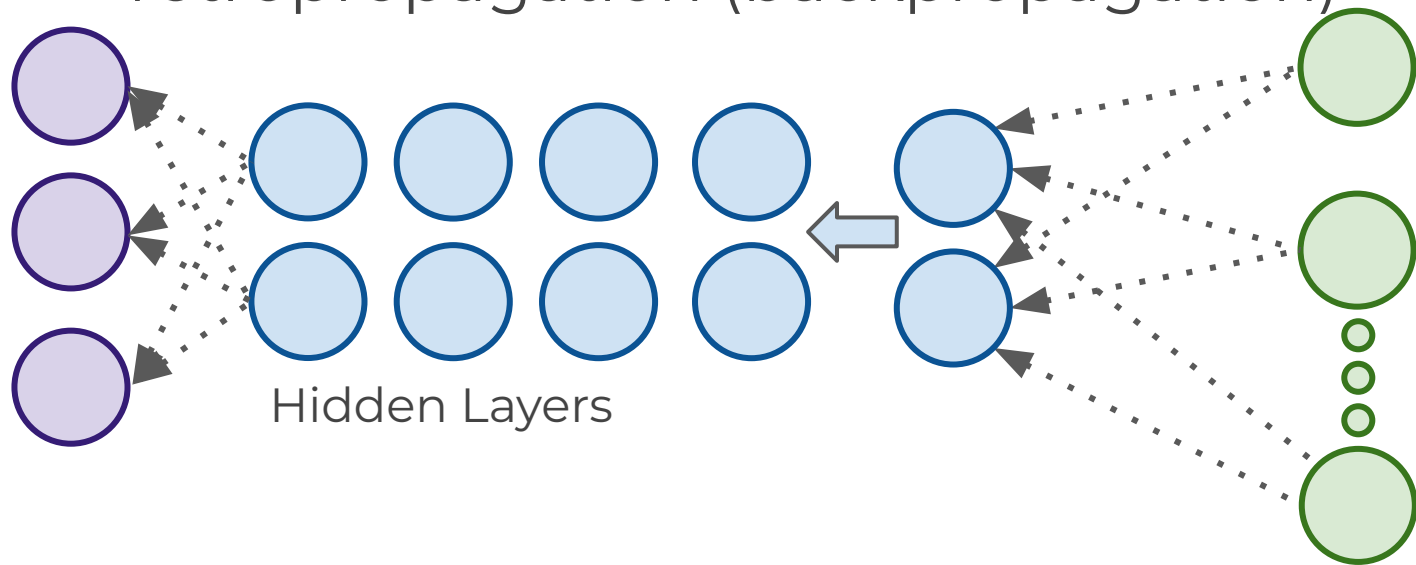
- Pour les données complexes, nous avons besoin de réseaux profonds





Deep Learning

- Des problèmes peuvent survenir lors de la rétropropagation (backpropagation)





Deep Learning

- La backpropagation va à l'envers de la couche de sortie vers la couche d'entrée, en propageant le gradient d'erreur.
- Pour les réseaux plus profonds, les problèmes peuvent provenir de la backpropagation, des vanishing et exploding gradients !





Deep Learning

- En remontant vers les couches "inférieures", les gradients deviennent souvent plus petits, ce qui fait que les poids ne changent jamais aux niveaux inférieurs.
- L'inverse peut aussi se produire, des gradients explosent au retour, causant des problèmes.





Deep Learning

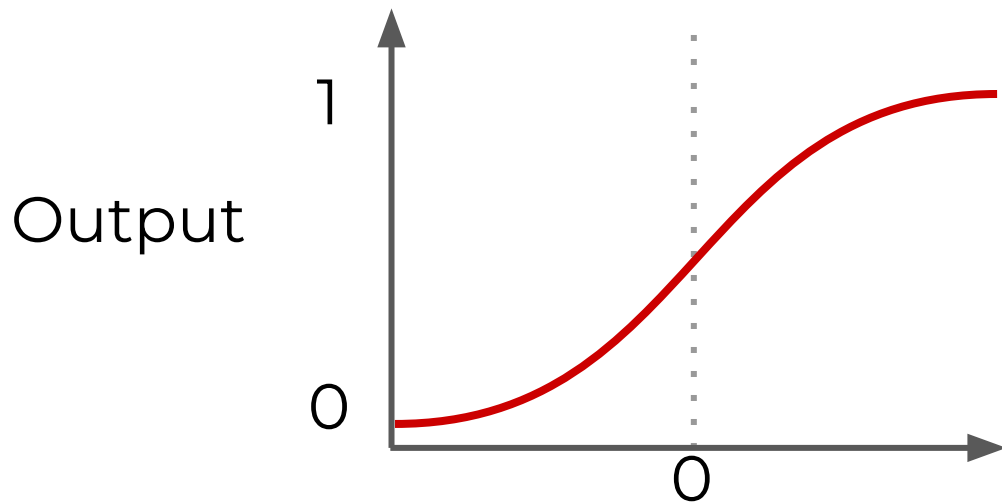
- Voyons pourquoi cela pourrait se produire et comment nous pouvons y remédier.
- Ensuite, dans la prochaine vidéo, nous discuterons de la manière dont ces problèmes affectent spécifiquement le RNN et comment utiliser le LSTM et le GRU pour les résoudre.





Deep Learning

- Pourquoi cela se produit-il ?



$$f(x) = \frac{1}{1 + e^{-(x)}}$$

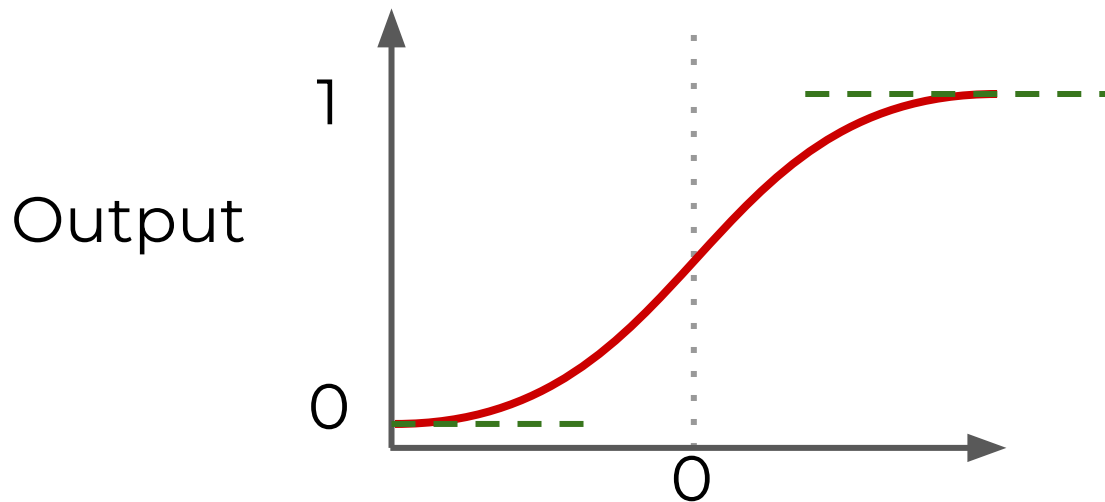
$$z = wx + b$$





Deep Learning

- Pourquoi cela se produit-il ?



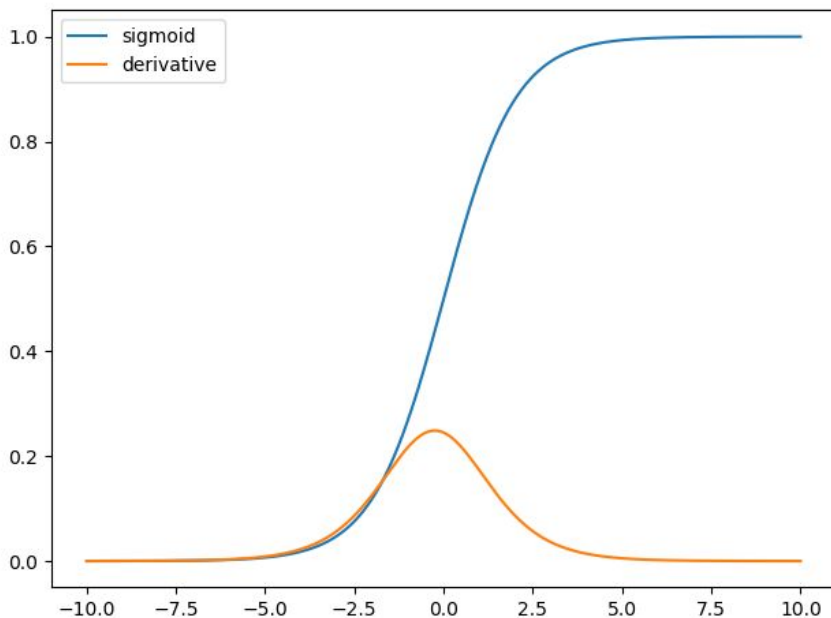
$$f(x) = \frac{1}{1 + e^{-(x)}}$$





Deep Learning

- La dérivée peut être beaucoup plus petit !





Deep Learning

- Lorsque **n** couches cachées utilisent une activation comme la fonction sigmoïde, **n** petites dérivées sont multipliées ensemble.
- Le gradient pourrait diminuer de façon exponentielle à mesure que nous nous propageons vers les couches initiales.

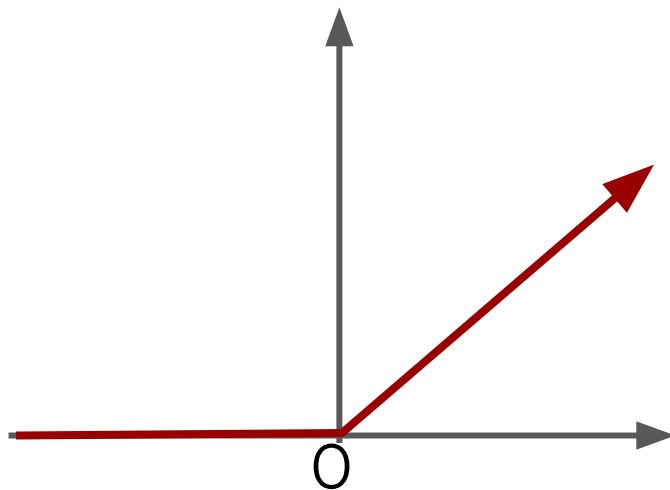




Deep Learning

- Utilisation de différentes fonctions d'activation

Output



$$z = wx + b$$

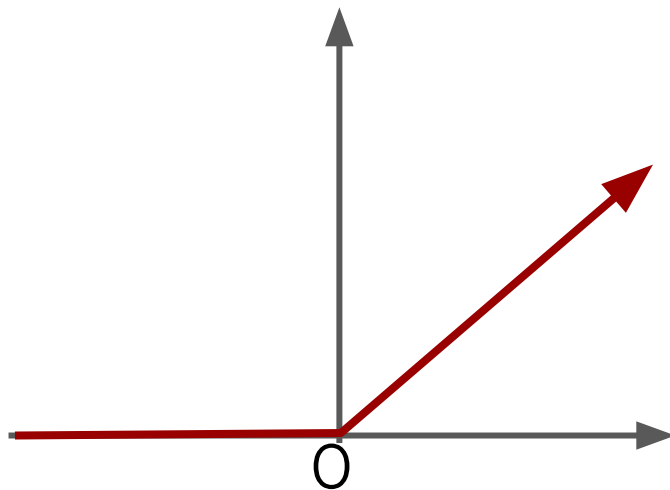




Deep Learning

- La ReLu ne sature pas les valeurs positives.

Output



$$z = wx + b$$

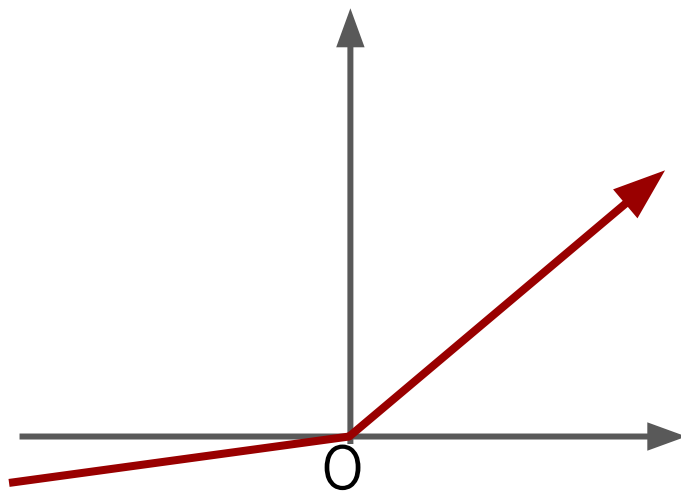




Deep Learning

- “Leaky” ReLU

Output



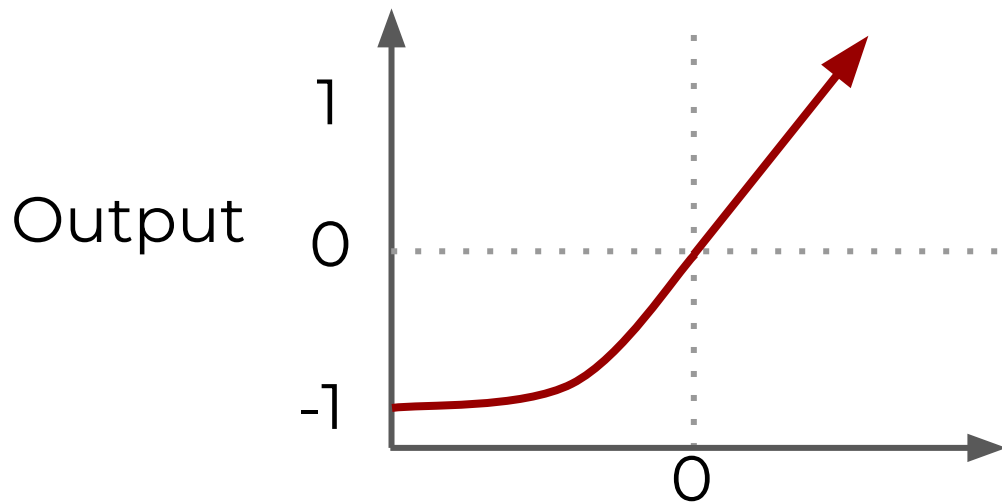
$$z = wx + b$$





Deep Learning

- Unité Linéaire Exponentielle (ELU)



$$z = wx + b$$





Deep Learning

- Une autre solution consiste à effectuer une normalisation par batch (lot), où votre modèle normalisera chaque batch en utilisant la moyenne et l'écart-type du batch.





Deep Learning

- Le choix d'une initialisation différente des poids peut également contribuer à atténuer ces problèmes (initialisation Xavier).





Deep Learning

- Outre la normalisation par batch, les chercheurs ont également utilisé le "gradient clipping", qui consiste à couper les gradients avant qu'ils n'atteignent une limite prédéterminée (par exemple, les gradients coupés doivent être compris entre -1 et 1)





Deep Learning

- RNN pour Time Series présentent leurs propres défis de gradient, explorons les unités de neurones spéciales LSTM (Long Short Term Memory) qui aident à résoudre ces problèmes !



Unités LSTM et GRU



MonCoachData



Deep Learning

- De nombreuses solutions présentées précédemment pour les vanishing gradients peuvent également s'appliquer au RNN : différentes fonctions d'activation, normalisations par batch, etc...
- Toutefois, en raison de la longueur des séries temporelles, celles-ci pourraient ralentir l'entraînement





Deep Learning

- Une solution possible serait de raccourcir les pas de temps utilisés pour les prévisions, mais cela rend le modèle moins apte à prédire les tendances à long terme.





Deep Learning

- Un autre problème rencontré par un RNN est qu'après un certain temps, le réseau commencera à "oublier" les premières entrées, car l'information est perdue à chaque étape passant par le RNN.
- Nous avons besoin d'une sorte de "mémoire à long terme" pour nos réseaux.





Deep Learning

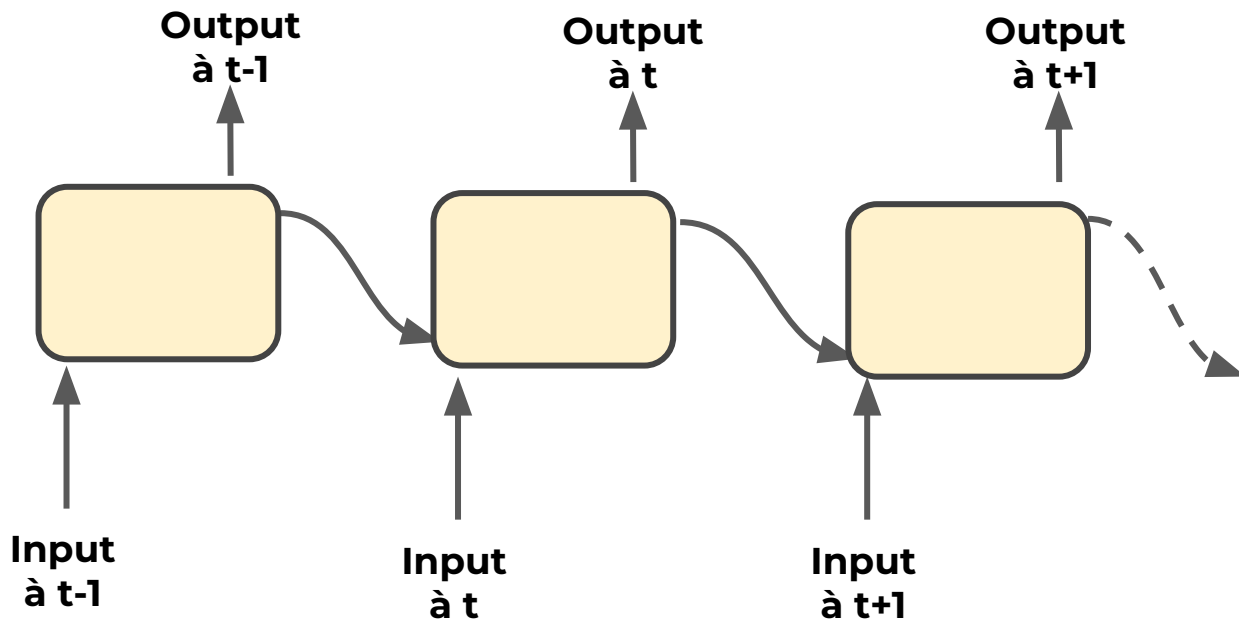
- La cellule LSTM (Long Short-Term Memory) a été créée pour aider à résoudre ces problèmes du RNN.
- Voyons comment fonctionne une cellule LSTM !





Deep Learning

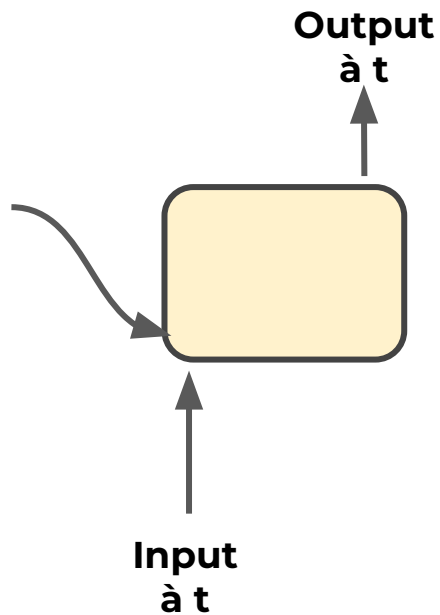
- Un RNN typique





Deep Learning

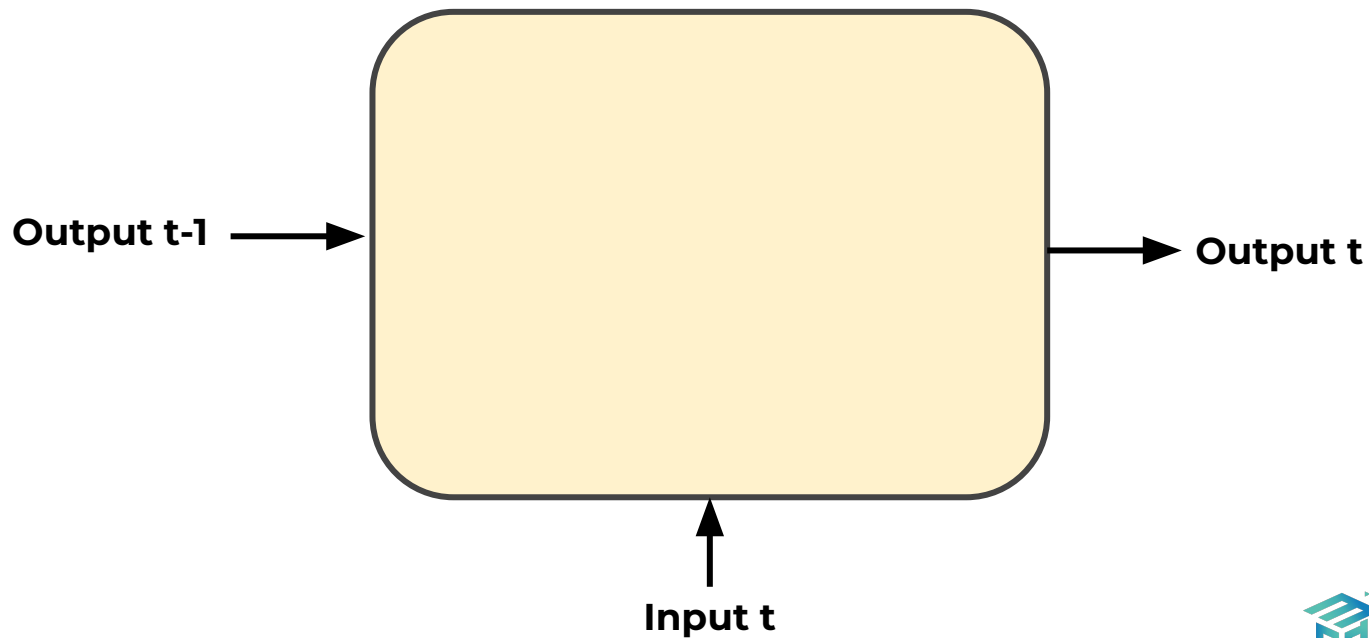
- RNN





Deep Learning

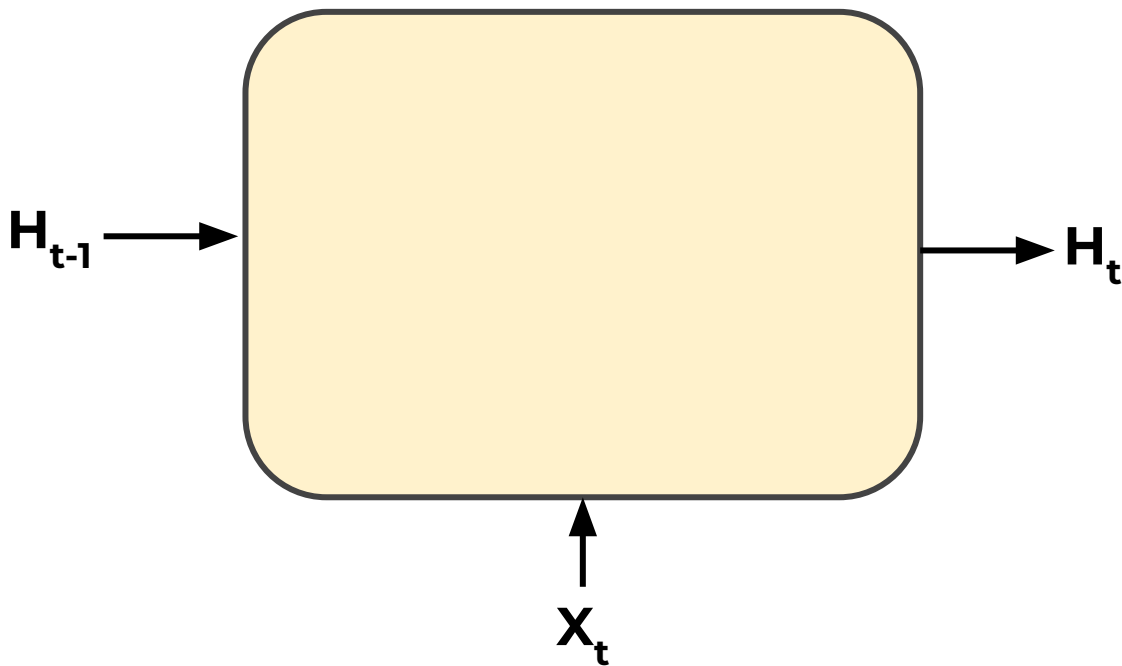
- RNN





Deep Learning

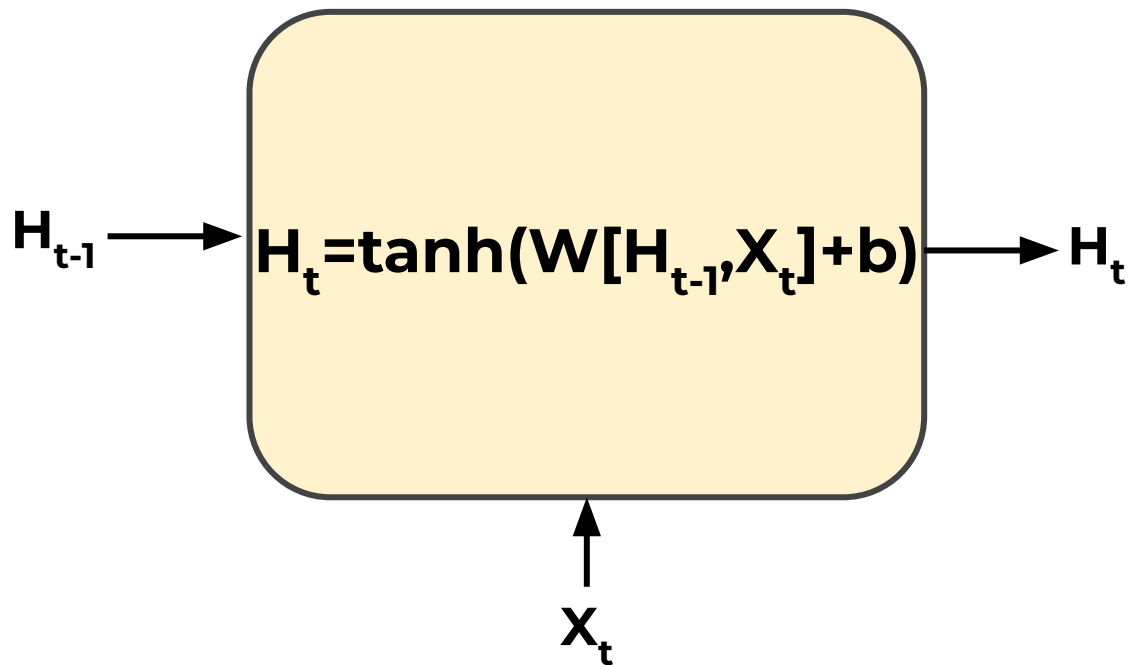
- RNN





Deep Learning

- RNN





Deep Learning

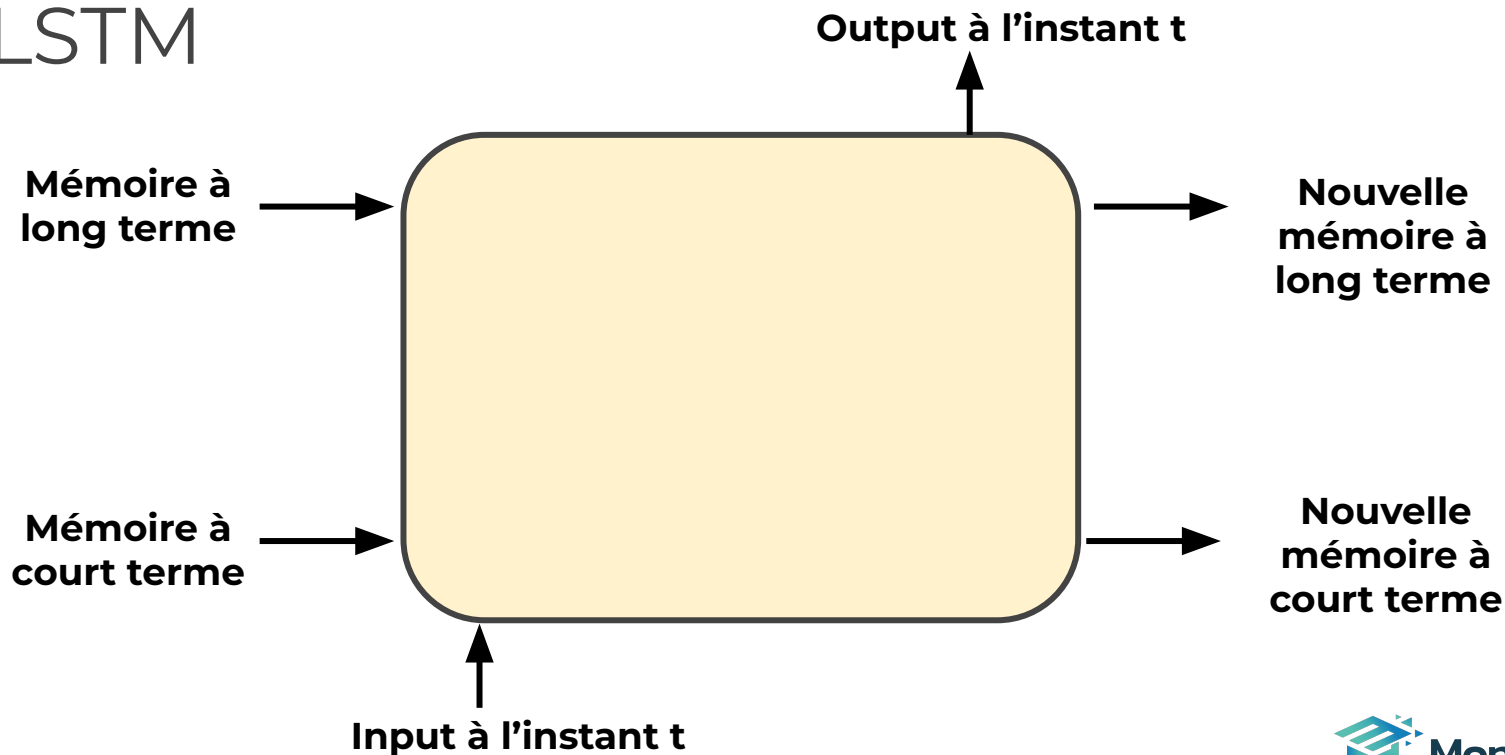
- LSTM





Deep Learning

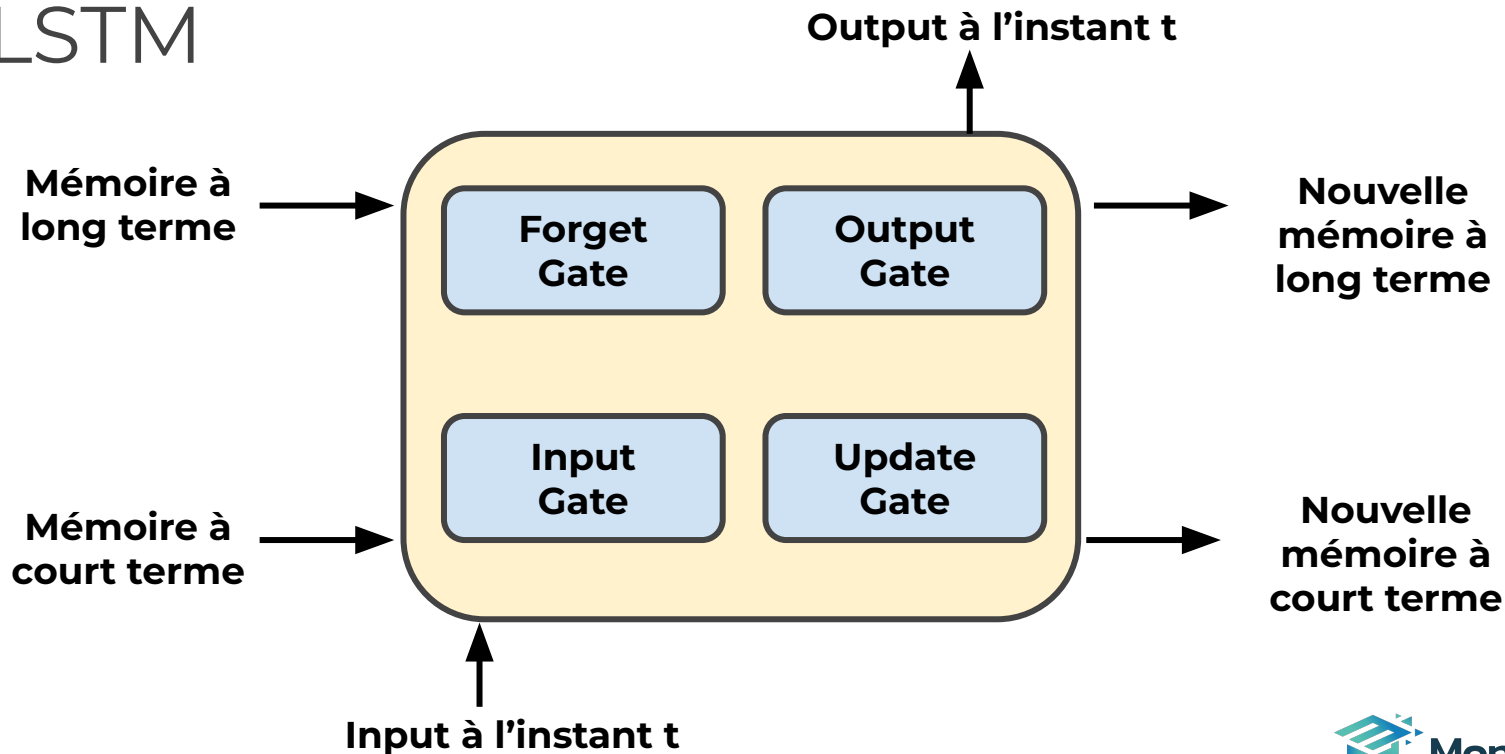
- LSTM





Deep Learning

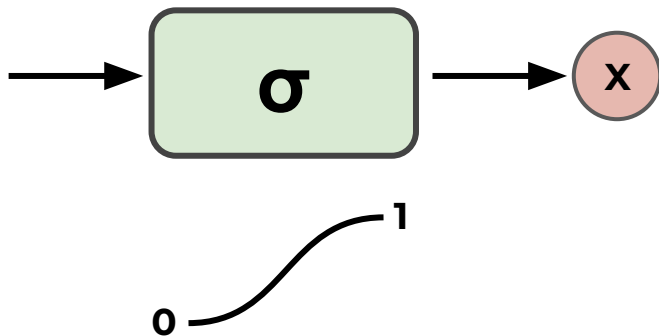
- LSTM





Deep Learning

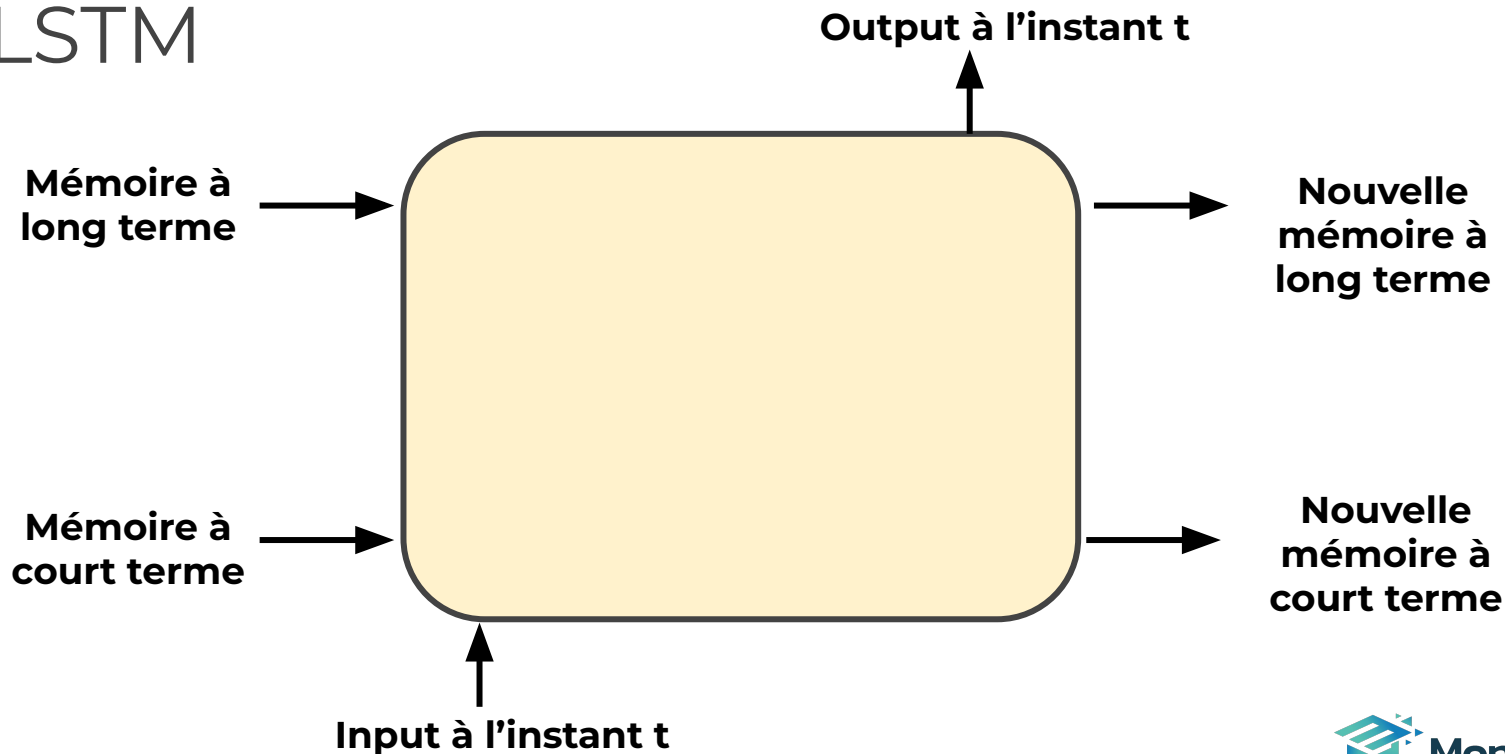
- Les portes (gates) laissent éventuellement passer des informations





Deep Learning

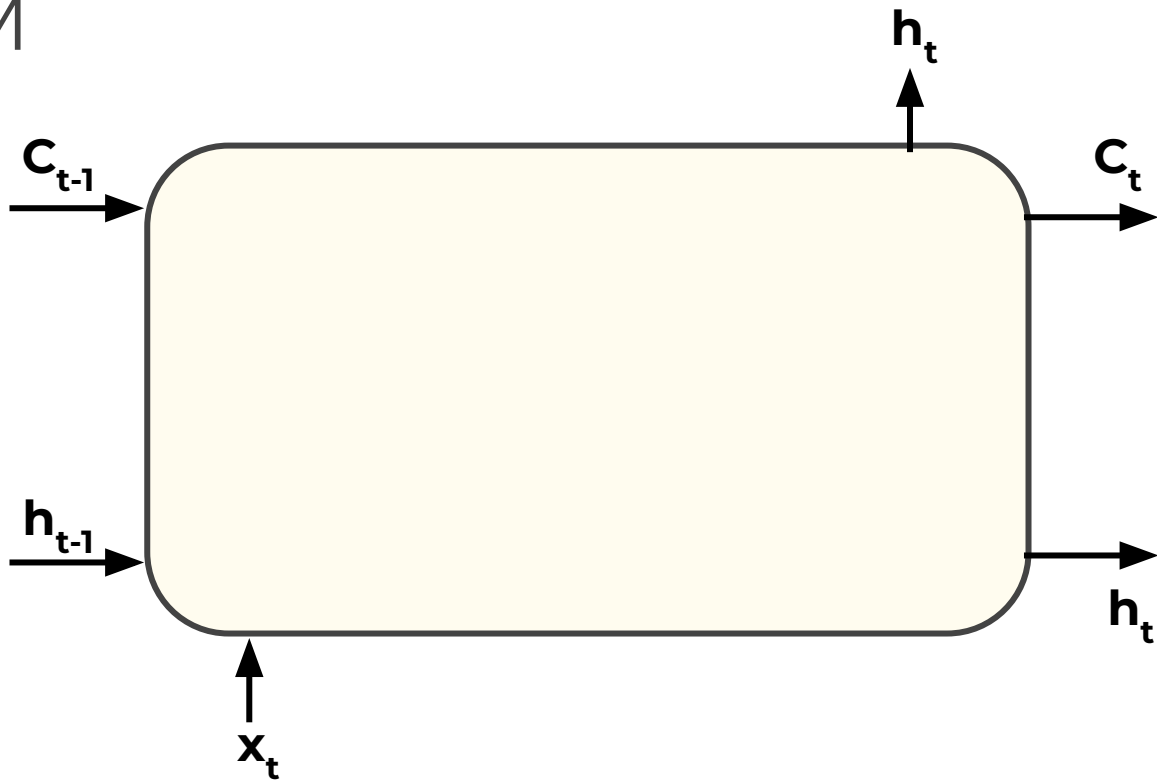
- LSTM





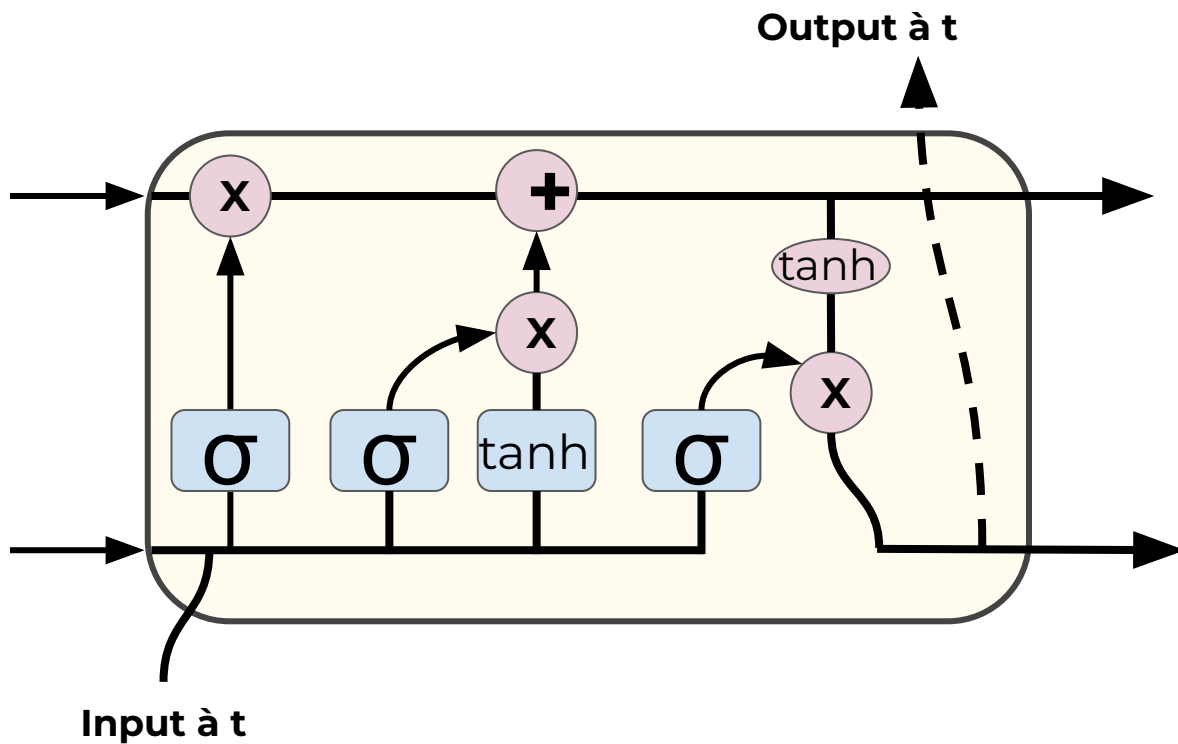
Deep Learning

- LSTM





Une cellule LSTM



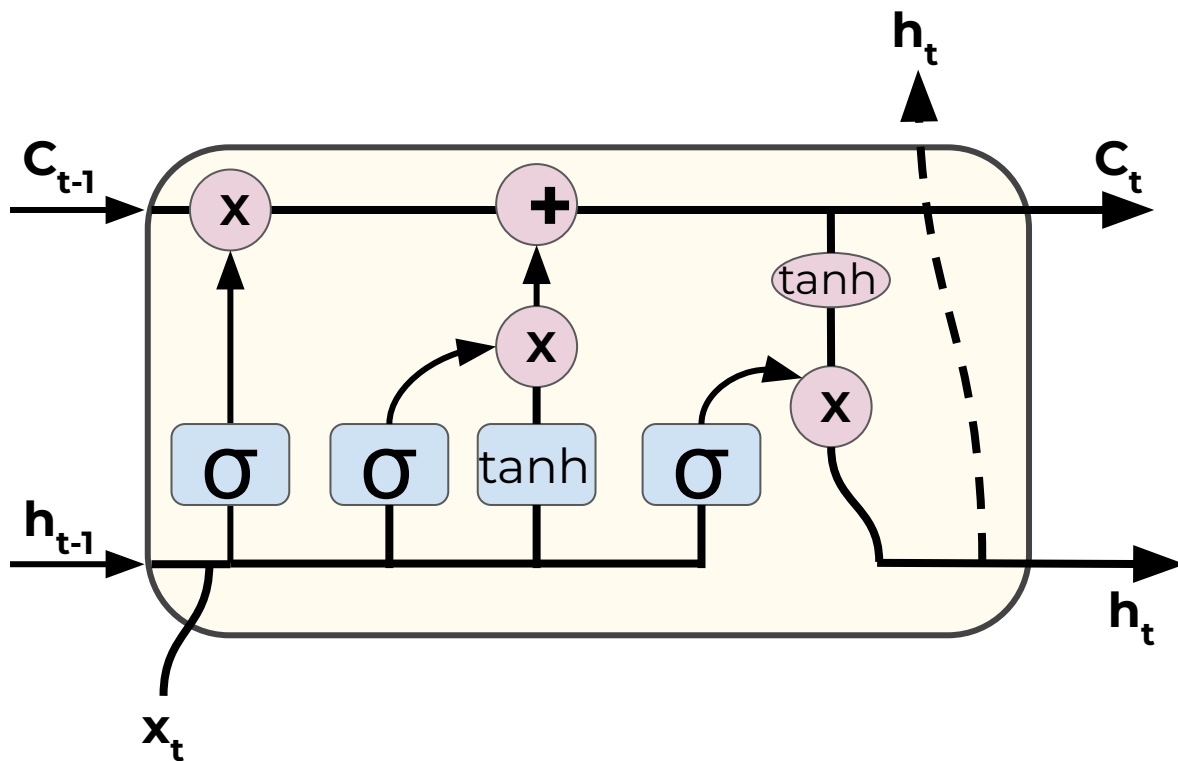
Ici, nous pouvons voir la cellule LSTM dans son intégralité.

Passons en revue le processus !





Une cellule LSTM



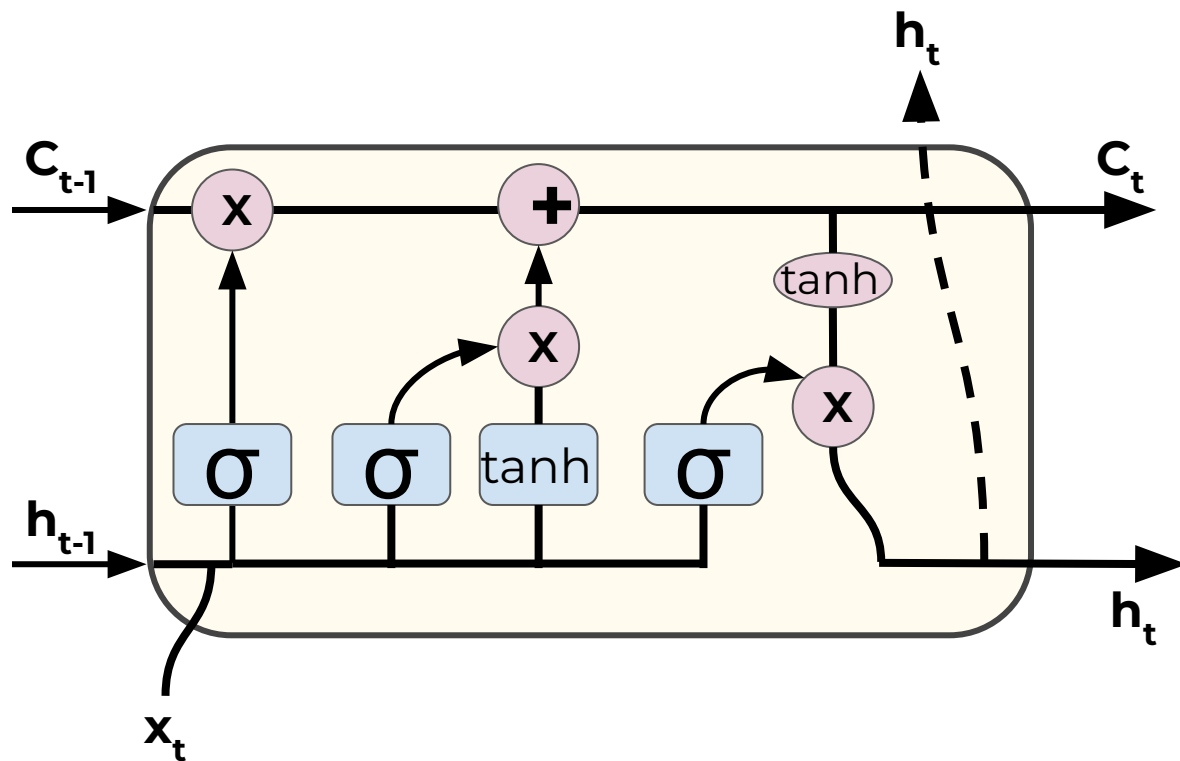
Ici, nous pouvons voir la cellule LSTM dans son intégralité.

Passons en revue le processus !



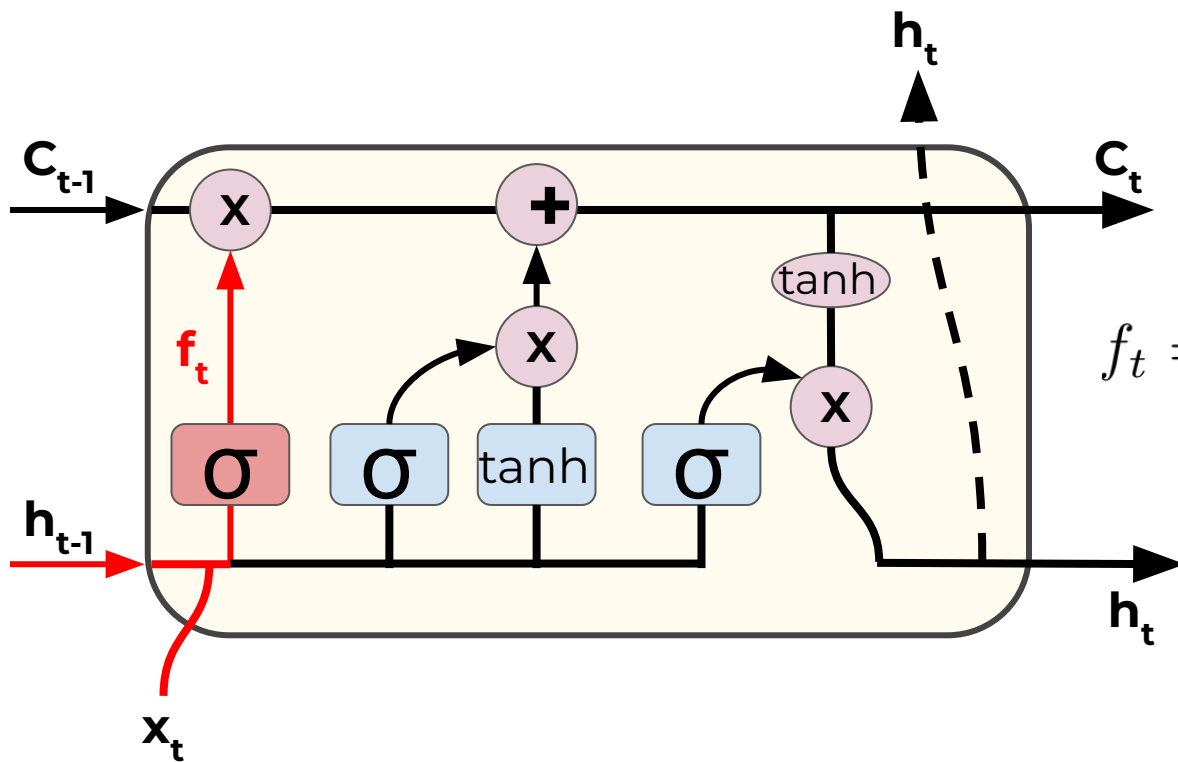


Une cellule LSTM





Une cellule LSTM

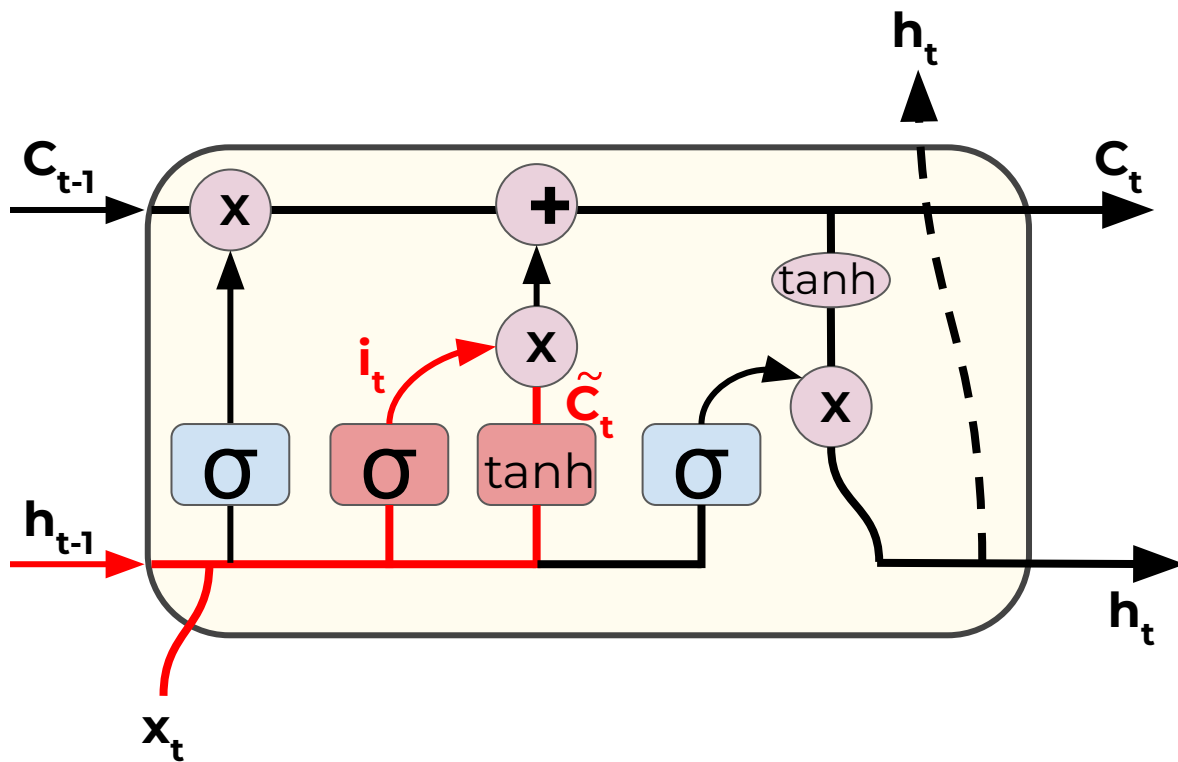


$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$





Une cellule LSTM

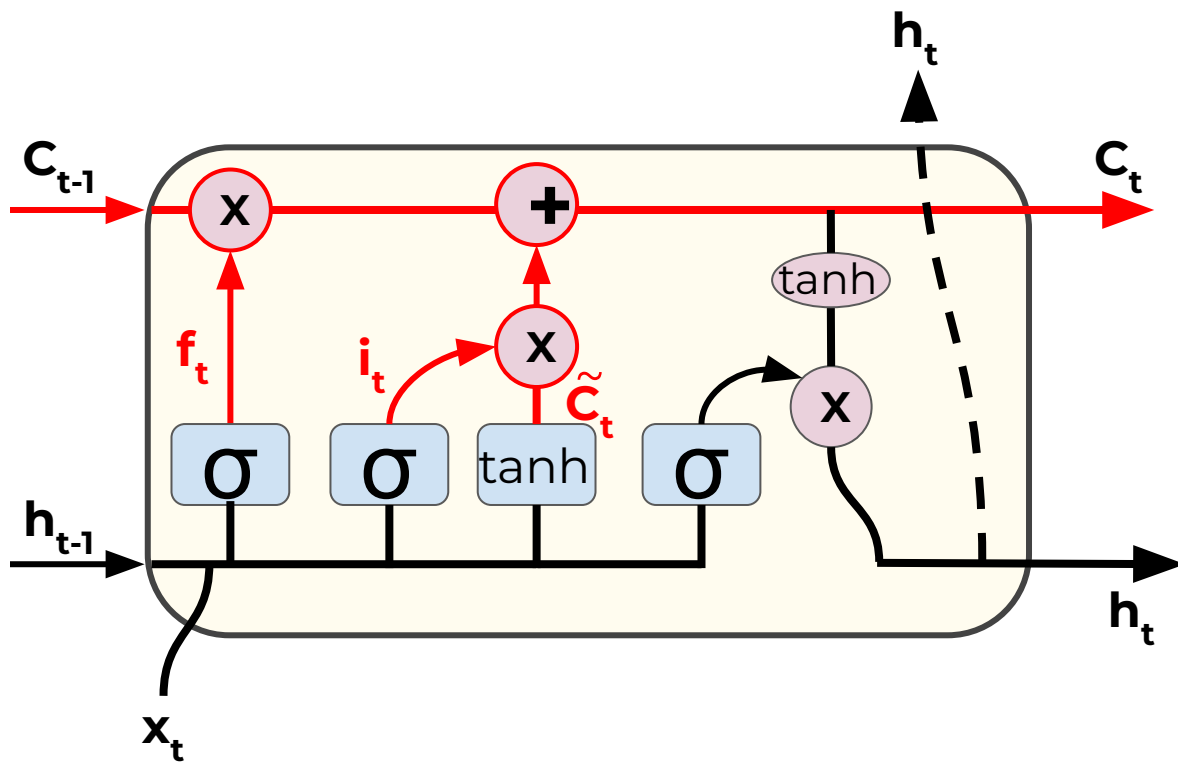


$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$





Une cellule LSTM

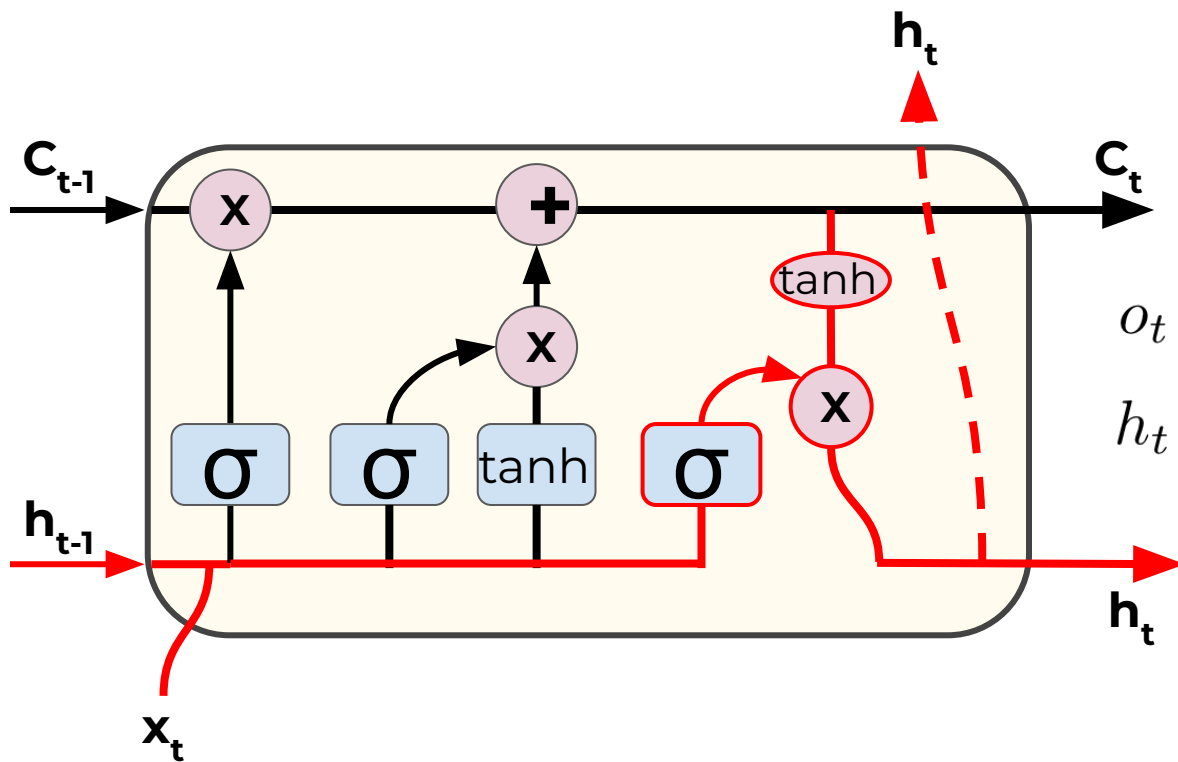


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$





Une cellule LSTM



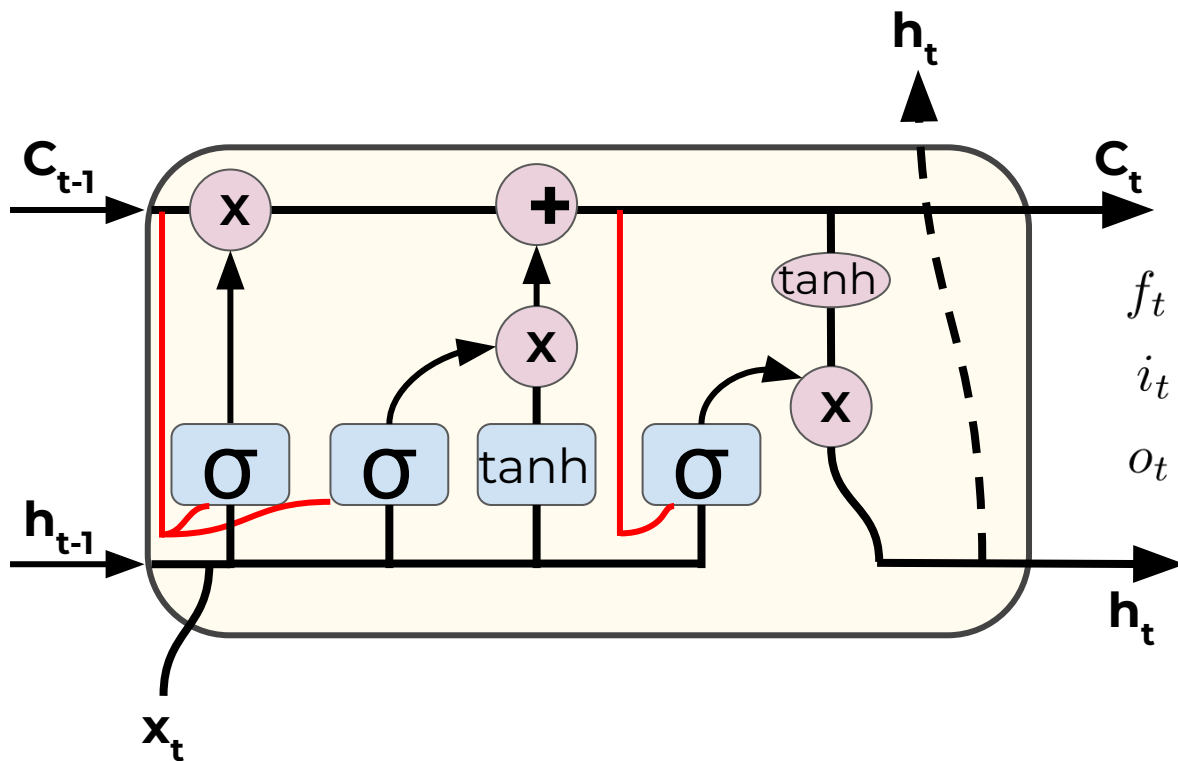
$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$





Une cellule LSTM avec “peepholes”



$$f_t = \sigma(W_f \cdot [C_{t-1}, h_{t-1}, x_t] + b_f)$$

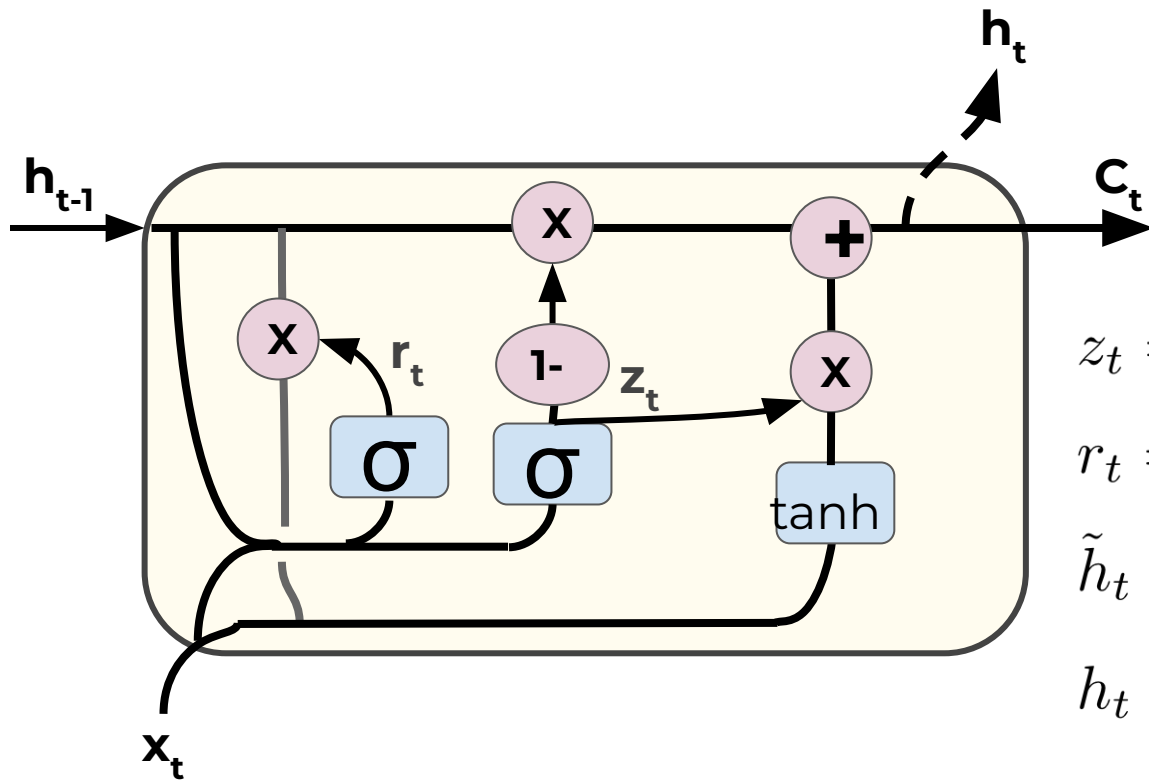
$$i_t = \sigma(W_i \cdot [C_{t-1}, h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma(W_o \cdot [C_t, h_{t-1}, x_t] + b_o)$$





Unités Récurrentes à Portes (GRU)



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$





Deep Learning

- Heureusement, grâce à notre bibliothèque python de Deep Learning, nous avons simplement besoin d'appeler l'importation pour RNN ou LSTM au lieu de devoir coder tout cela nous-mêmes !
- Voyons comment utiliser les LSTMs avec du code Python !



RNN de base





Deep Learning

- Voyons maintenant comment utiliser un RNN sur une série temporelle de base, comme une onde sinusoïdale.
- Avant de passer sur le notebook, voyons rapidement à quoi ressemblent les lots de séquences RNN.





Deep Learning

- Imaginons une simple série temporelle :
 - **[0,1,2,3,4,5,6,7,8,9]**
- Nous séparons cela en deux parties :
 - **[0,1,2,3,4,5,6,7,8]** **[9]**
- Etant donné une **séquence** **d'entraînement**, prédire la **valeur de la séquence suivante**.





Deep Learning

- Gardez à l'esprit que nous pouvons généralement décider de la durée de la séquence d'entraînement et de l'étiquette (label) prédite :
 - **[0,1,2,3,4]** **[5,6,7,8,9]**





Deep Learning

- Nous pouvons également modifier la taille du point d'entraînement, ainsi que le nombre de séquences à alimenter par batch :
 - **[0,1,2,3]** **[4]**
 - **[1,2,3,4]** **[5]**
 - **[2,3,4,5]** **[6]**





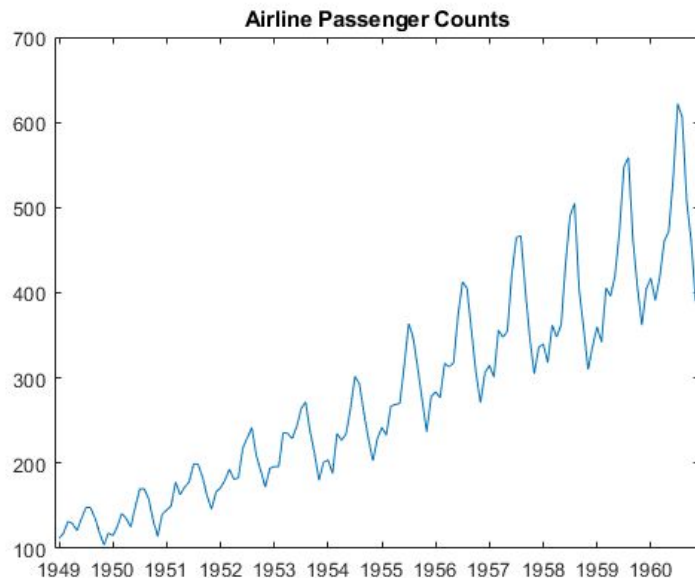
- Alors, comment décider de la durée de la séquence d'entraînement ?
 - Il n'y a pas de réponse définitive, mais elle devrait être au moins suffisamment longue pour saisir toute information utile sur les tendances.





Deep Learning

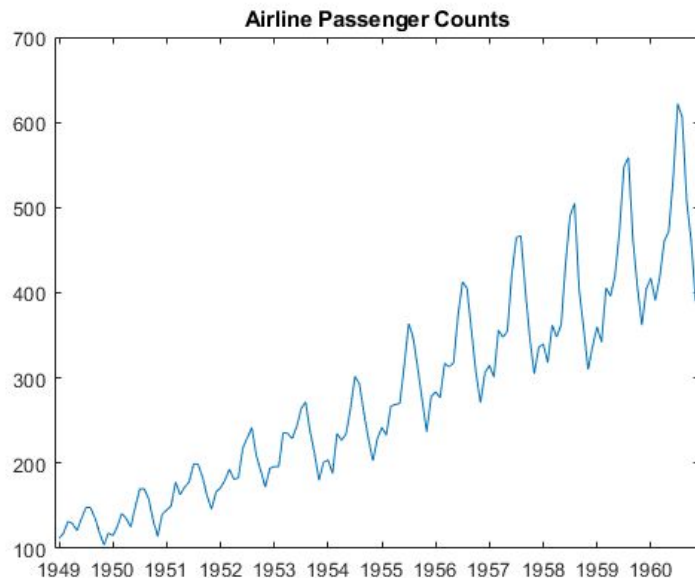
- Par exemple, s'il s'agit de données saisonnières :





Deep Learning

- Si c'est mensuel, nous devons inclure au moins 12 mois dans la séquence d'entraînement





Deep Learning

- Cela nécessite souvent une connaissance et une expérience du domaine, ainsi qu'une simple expérimentation et l'utilisation de la RMSE pour mesurer l'erreur des prévisions.
- En général, un bon choix de départ pour l'étiquette (label) n'est qu'un point de données vers l'avenir.





Deep Learning

- Comment faire des prévisions avec les RNNs ?
 - Imaginons que toutes nos données le soient :
 - **[0,1,2,3,4,5,6,7,8,9]**
 - Et nous entraînons sur des séquences telles que :
 - **[0,1,2,3]** → **[4]**
 - **[1,2,3,4]** → **[5]**
 - **[2,3,4,5]** → **[6]**
-



Deep Learning

- Notre technique de prévision consiste alors à prévoir une unité de temps à l'avance, puis à incorporer notre prévision dans la séquence suivante que nous prévoyons au départ.
- Passons en revue un exemple rapide !





Deep Learning

- Comment faire des prévisions avec les RNNs ?
 - Imaginons que toutes nos données le soient :
 - **[0,1,2,3,4,5,6,7,8,9]**
 - Et nous entraînons sur des séquences telles que :
 - **[0,1,2,3]** **[4]**
 - **[1,2,3,4]** → **[5]**
 - **[2,3,4,5]** → **[6]**
-



Deep Learning

- **[6,7,8,9]** → **[10]** Prévission !





Deep Learning

- **[6,7,8,9]** $\Rightarrow$ **[10]** Prédiction !
- Ensuite continuer à faire des prévisions :





Deep Learning

- **[6,7,8,9]** $\Rightarrow$ **[10]** Prédiction !
 - Ensuite continuer à faire des prévisions :
 - **[7,8,9,10]** $\Rightarrow$ **[11.2]**





Deep Learning

- **[6,7,8,9]** $\Rightarrow$ **[10]** Prédiction !
 - Ensuite continuer à faire des prévisions :
 - **[7,8,9,10]** $\Rightarrow$ **[11.2]**
 - **[8,9,10,11.2]** $\Rightarrow$ **[12.4]**





Deep Learning

- **[6,7,8,9]** → **[10]** Prédiction !
 - Ensuite continuer à faire des prévisions :
 - **[7,8,9,10]** → **[11.2]**
 - **[8,9,10,11.2]** → **[12.4]**
 - **[9,10,11.2,12.4]** → **[14]**





Deep Learning

- **[6,7,8,9]** $\Rightarrow$ **[10]** Prédiction !
 - Ensuite continuer à faire des prévisions :
 - **[7,8,9,10]** $\Rightarrow$ **[11.2]**
 - **[8,9,10,11.2]** $\Rightarrow$ **[12.4]**
 - **[9,10,11.2,12.4]** $\Rightarrow$ **[14]**
 - **[10,11.2,12.4,14]** $\Rightarrow$ Prédiction réalisée





Deep Learning

- Allons plus loin avec Python !



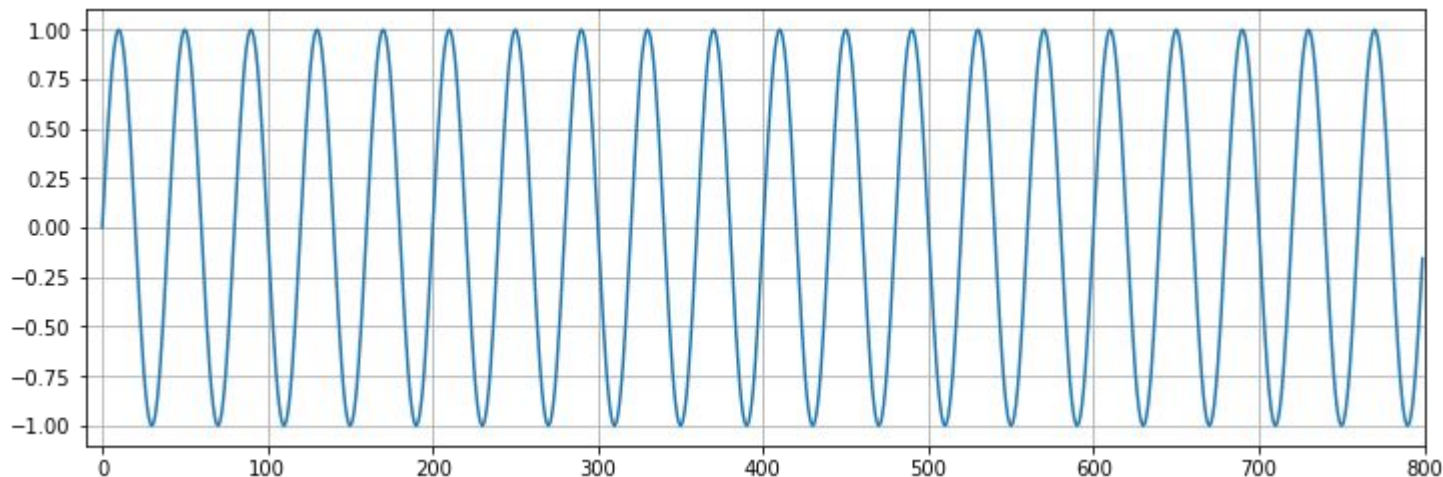
RNN de base sur une onde sinusoïdale





Deep Learning

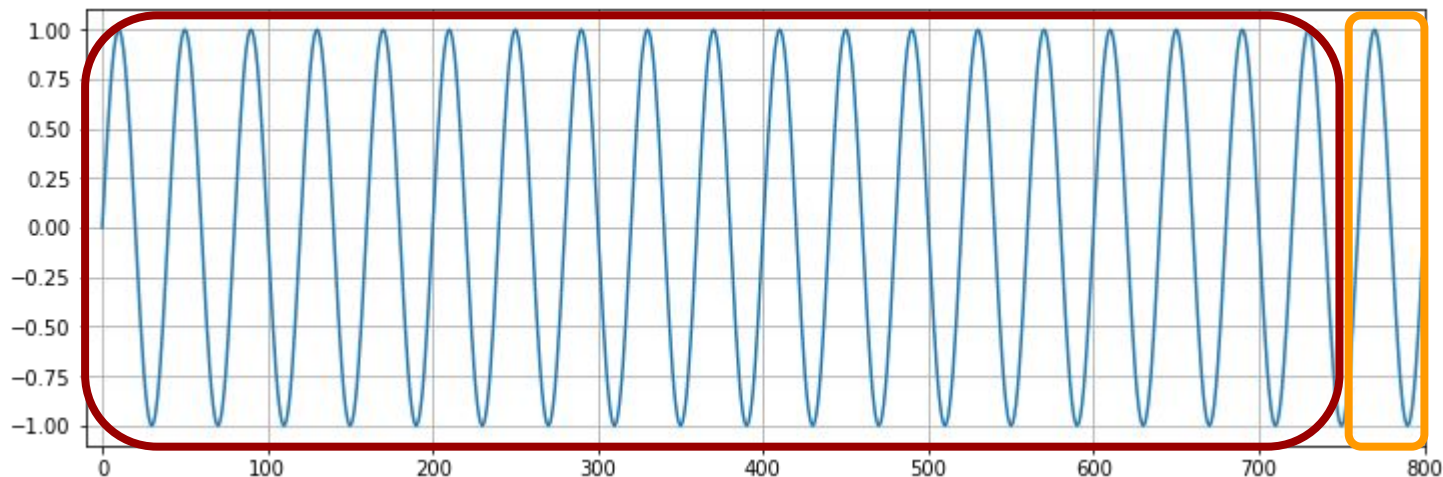
- Entraînons et évaluons maintenant notre RNN
- Rappelez-vous nos données d'origine :





Deep Learning

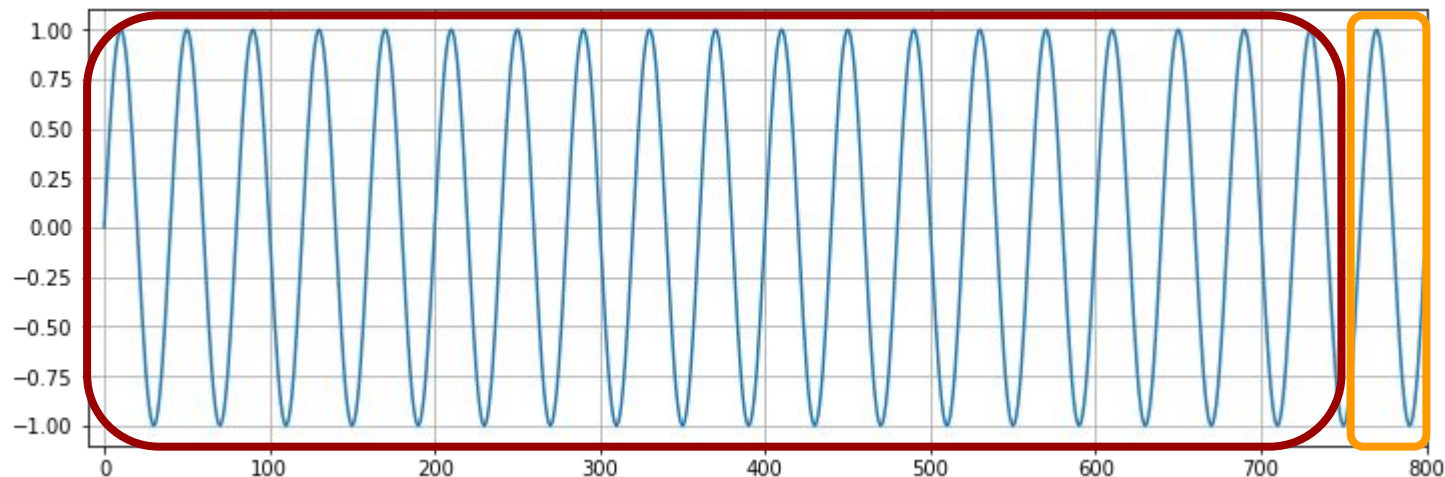
- Nous séparons cela en **train_set** et **test_set** :





Deep Learning

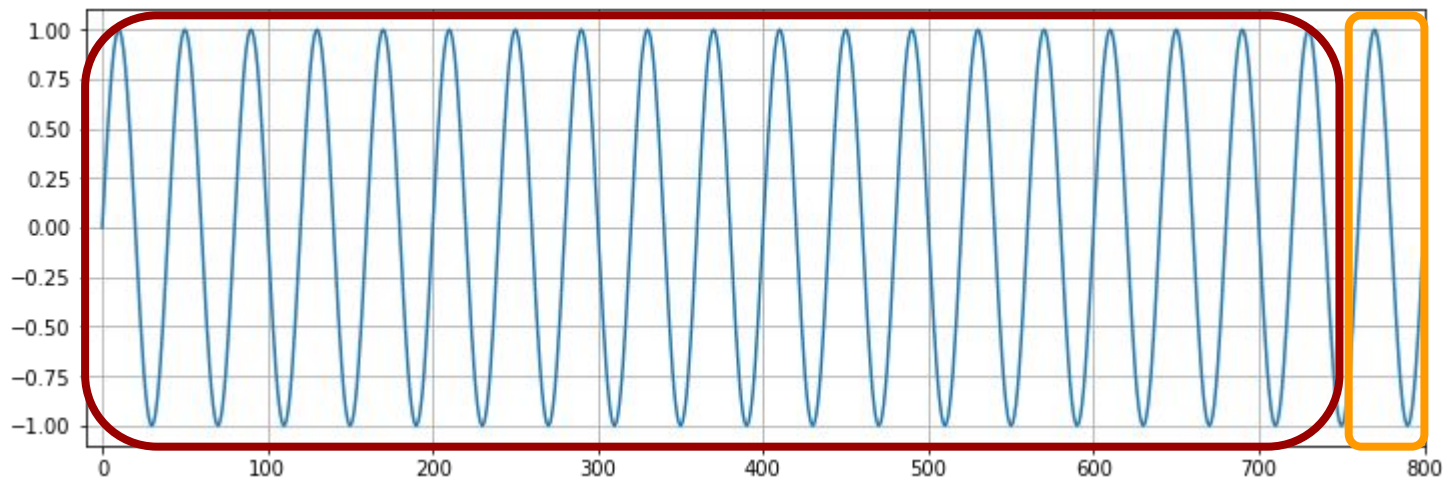
- Pendant l'entraînement, nous avons pu évaluer les performances sur la base des données de test.





Deep Learning

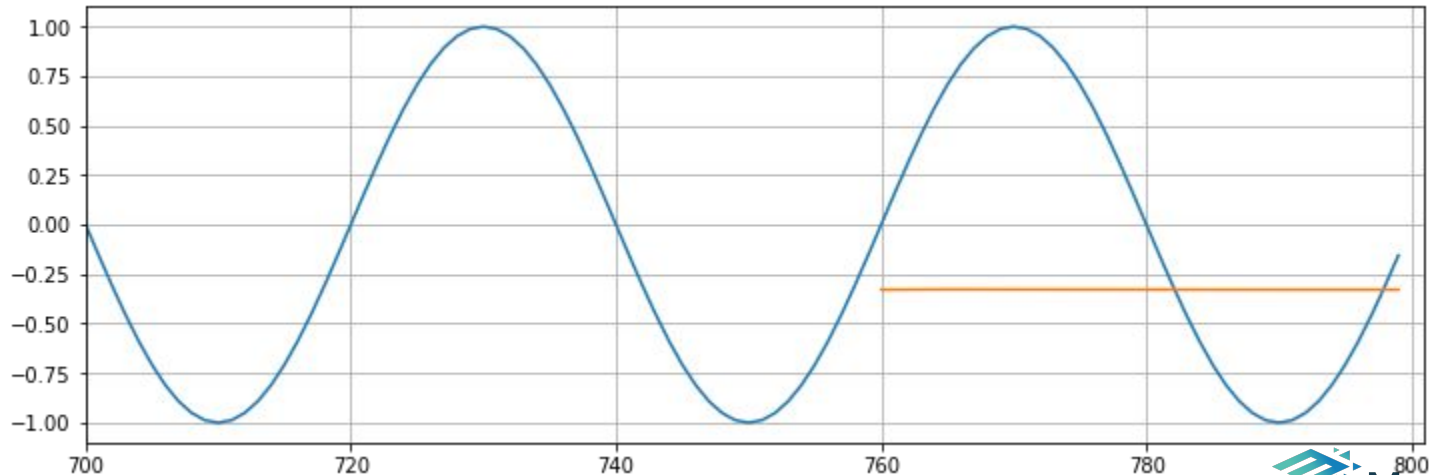
- Zoomons sur cette portée !





Deep Learning

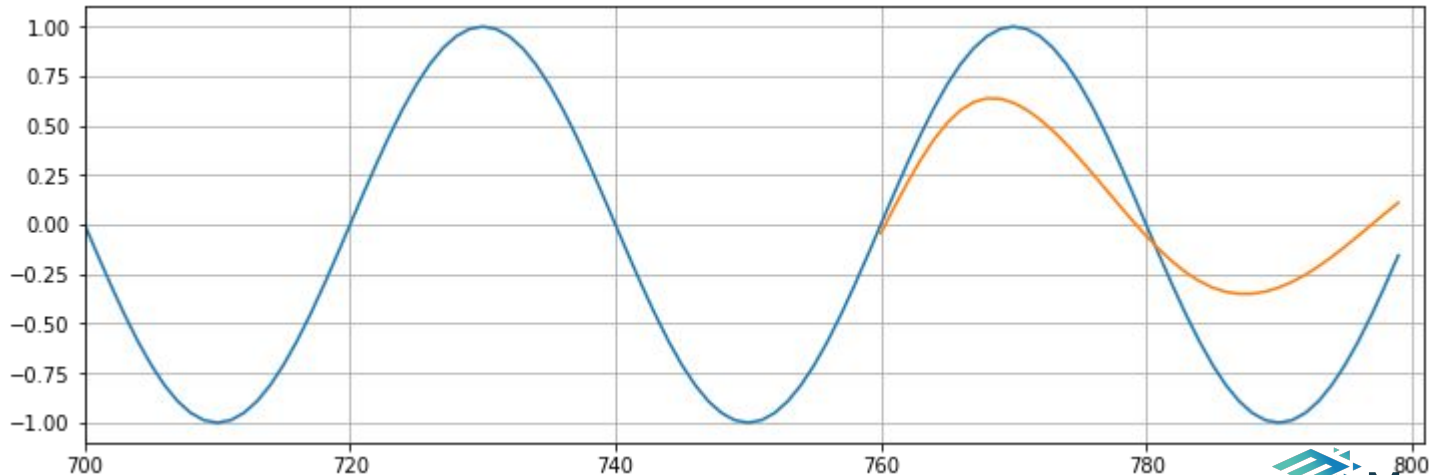
- Au fur et à mesure que nous entraînons, nous ferons des prévisions sur cette gamme pour voir visuellement l'entraînement.





Deep Learning

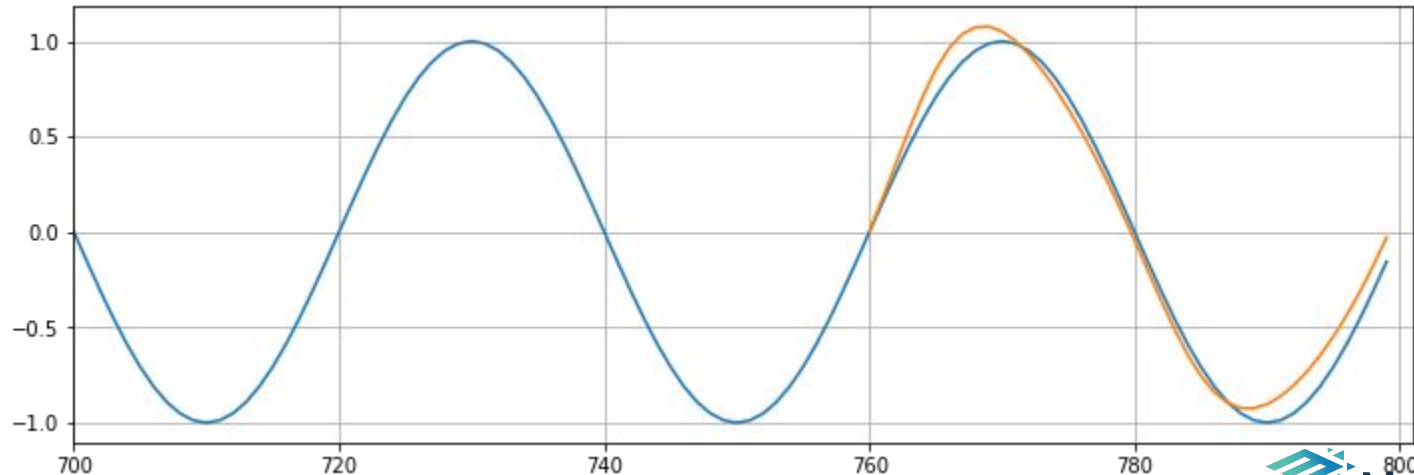
- Au fur et à mesure que nous entraînons, nous ferons des prévisions sur cette gamme pour voir visuellement l'entraînement.





Deep Learning

- Au fur et à mesure que nous entraînons, nous ferons des prévisions sur cette gamme pour voir visuellement l'entraînement.





Deep Learning

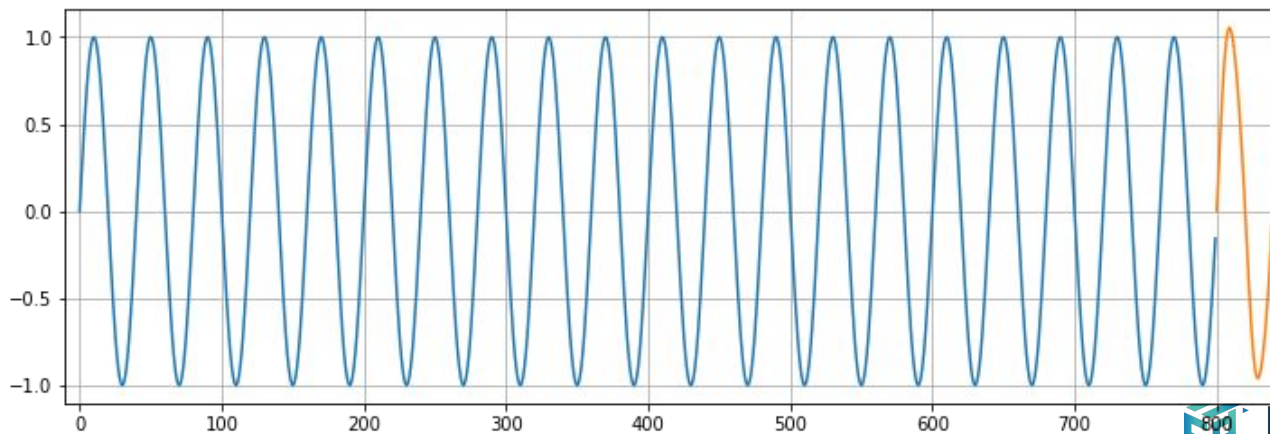
- Finalement, lorsque nous serons satisfaits des résultats de la portion test, il sera temps de faire des prévisions dans un futur inconnu.
- Cela signifie qu'il faut entraîner de nouveau sur toutes les données disponibles (entraînement et test) et prévoir au-delà de la portée des données d'origine.





Deep Learning

- Gardez à l'esprit que pour des données réelles, nous n'aurons plus de valeurs auxquelles comparer ces prévisions !



RNN sur une Time Series



MonCoachData



Exercices RNN



MonCoachData



Solutions Exercices RNN



Time Series multivariées avec des LSTM RNNs





Deep Learning

- En prime, nous discuterons de la façon d'utiliser les LSTMs et les RNNs pour prédire les séries temporelles multivariées.
- Gardez à l'esprit qu'il y a quelques inconvénients à utiliser les LSTMs pour cette approche !





Deep Learning

- Comme pour tous les réseaux de neurones, le modèle est essentiellement une boîte noire, difficile à interpréter.
- Il existe également de nombreuses alternatives bien étudiées qui sont plus simples, comme les modèles SARIMAX et VARMAX.





Deep Learning

- Nous vous recommandons vivement d'essayer ces approches plus conventionnelles avant de vous décider pour les LSTMs ou les RNNs pour les données de séries temporelles multivariées.
- Heureusement, la mise en place de données multivariées ne nécessite que deux changements principaux.





Deep Learning

- Time Series multivariées:
 - Modifier la forme de l'entrée dans la couche LSTM pour refléter la structure 2-D
 - La couche dense finale doit avoir un neurone par feature/variable.
- Examinons ces changements !

